

TUGAS 7 : MAIN MEMORY



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Nabila Senja Ramadhani (2410651043)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

Contents

DAFTAR ISI	2
BAB I	4
1.1 Latar Belakang	4
1.2 Tujuan Praktikum	4
1.3 Spesifikasi Sistem.....	5
BAB II	6
2.1 Memory Management.....	6
2.2 Contiguous Allocation	7
2.3 Paging	9
2.4 Page Table Structures	10
2.5 Swapping.....	11
2.6 Architecture Analysis	12
BAB III.....	14
3.1 Analaisis Memory Layout.....	14
1. Deskripsi Tugas	14
2. Source Code	14
3. Screenshot Output.....	17
4. Penjelasan Implementasi.....	17
3.2 Memory Allocator	19
3.2.1 Deskripsi Tugas	19
3.2.2 Source Code.....	20
3.2.3 Screenshot Output.....	26
3.2.4 Penjelasan Implementasi	26
3.3 Advanced Paging System.....	28

3.3.1	Deskripsi Tugas	28
3.3.2	Source Code.....	29
3.3.3	Screenshot Output.....	34
3.3.4	Penjelasan Implementasi	35
3.4	Multi-Level Page Table.....	36
3.4.1	Deskripsi Tugas	36
3.4.1	Screenshot Output.....	37
3.4.1	Source Code.....	37
3.4.1	Penjelasan Implementasi	41
3.5	Swapping Mechanism	43
3.5.1	Deskripsi Tugas	43
3.5.2	Source Code.....	44
3.5.3	Screenshot Output.....	49
3.5.4	Penjelasan Implementasi	50
3.6	Architecture Analysis	51
3.6.1	Deskripsi Tugas	51
3.6.2	Screenshot Output.....	51
3.6.3	Penjelasan Implementasi	52
BAB IV		54

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan sistem operasi modern menuntut pengelolaan memori yang efisien dan andal untuk mendukung eksekusi banyak proses secara bersamaan. Keterbatasan memori fisik, kebutuhan akan ruang alamat yang besar, serta tuntutan performa yang tinggi mendorong sistem operasi untuk menerapkan berbagai mekanisme manajemen memori, seperti memory layout, dynamic memory allocation, paging, multi-level page table, dan swapping.

Mekanisme-mekanisme tersebut berperan penting dalam memastikan bahwa setiap proses memperoleh ruang memori yang cukup tanpa mengorbankan stabilitas dan kinerja sistem. Arsitektur prosesor juga memiliki pengaruh besar terhadap implementasi manajemen memori. Perbedaan antara arsitektur 32-bit dan 64-bit menentukan ukuran address space, struktur page table, serta kemampuan sistem dalam menangani aplikasi berskala besar. Pada sistem modern berbasis arsitektur 64-bit, ruang alamat virtual yang sangat luas memungkinkan penerapan multi-level paging dan virtual memory secara lebih optimal. Melalui praktikum ini, dilakukan eksplorasi langsung terhadap konsep-konsep manajemen memori sistem operasi dengan pendekatan simulasi dan implementasi program. Praktikum mencakup analisis memory layout, perbandingan algoritma alokasi memori, simulasi page replacement, implementasi multi-level page table, serta mekanisme swapping dengan berbagai strategi. Dengan melakukan eksperimen langsung pada sistem nyata, mahasiswa diharapkan mampu memahami keterkaitan antara teori manajemen memori dan implementasinya pada sistem operasi modern.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah untuk memahami konsep dan mekanisme manajemen memori pada sistem operasi secara menyeluruh melalui implementasi dan analisis. Praktikum ini bertujuan untuk mempelajari struktur memory layout dan segmentasi memori, memahami cara kerja dynamic memory allocation beserta dampak fragmentasi, menganalisis algoritma page replacement dan pengaruhnya terhadap performa sistem, memahami penerapan multi-level page table dan inverted page table, serta mempelajari mekanisme swapping dan strategi

pemilihan proses korban. Selain itu, praktikum ini juga bertujuan untuk mengaitkan teori manajemen memori dengan karakteristik arsitektur prosesor dan sistem yang digunakan.

1.3 Spesifikasi Sistem

Praktikum ini dijalankan pada sistem dengan spesifikasi perangkat keras dan konfigurasi sebagai berikut. Sistem menggunakan arsitektur x86_64 dengan prosesor AMD Ryzen 7 7435HS yang memiliki 8 core dan 16 thread. Prosesor ini mendukung mode operasi 32-bit dan 64-bit. Ukuran page memori yang digunakan oleh sistem adalah 4096 byte atau 4 KB. Hirarki cache terdiri dari L1 cache data dan instruksi masing-masing berukuran 256 KB per core, L2 cache sebesar 4 MB yang bersifat privat per core, serta L3 cache sebesar 16 MB yang bersifat shared antar core. Sistem operasi yang digunakan berbasis Linux dengan dukungan virtual memory dan multi-level paging.

BAB II

DASAR TEORI

2.1 Memory Management

Memory layout adalah pengaturan ruang alamat memori yang digunakan oleh suatu program selama proses eksekusi. Pada sistem operasi modern seperti Linux, setiap proses diberikan ruang alamat virtual yang kemudian dibagi menjadi beberapa segmen dengan fungsi yang berbeda. Pembagian ini bertujuan untuk mengelola memori secara efisien, menjaga kestabilan program, serta meningkatkan keamanan sistem.

Segmen text merupakan bagian memori yang menyimpan instruksi atau kode program yang akan dieksekusi oleh prosesor. Seluruh fungsi yang didefinisikan dalam program, seperti fungsi utama dan fungsi tambahan, ditempatkan pada segmen ini. Text segment bersifat read-only dan executable, artinya kode program tidak dapat diubah selama runtime namun tetap dapat dijalankan oleh CPU. Pembatasan ini dilakukan untuk mencegah modifikasi kode yang dapat menyebabkan kesalahan atau celah keamanan.

Segmen data digunakan untuk menyimpan variabel global dan variabel static yang digunakan oleh program. Segmen ini terbagi menjadi data yang telah diinisialisasi dan data yang belum diinisialisasi. Variabel global atau static yang memiliki nilai awal disimpan pada initialized data segment, sedangkan variabel yang tidak memiliki nilai awal disimpan pada area BSS dan secara otomatis diisi dengan nilai nol oleh sistem. String literal umumnya ditempatkan pada area data yang bersifat read-only untuk mencegah perubahan nilai secara tidak sengaja.

Heap merupakan bagian memori yang digunakan untuk alokasi memori dinamis selama program berjalan. Alokasi memori pada heap dilakukan menggunakan fungsi seperti `malloc`, `calloc`, atau `realloc`. Heap bersifat fleksibel karena ukuran memori yang digunakan dapat bertambah atau berkurang sesuai kebutuhan program. Pengelolaan memori heap menjadi tanggung jawab programmer, sehingga setiap alokasi harus diikuti dengan pembebasan memori untuk menghindari kebocoran memori. Pada sistem Linux, heap umumnya tumbuh ke arah alamat memori yang lebih tinggi.

Stack adalah segmen memori yang digunakan untuk menyimpan variabel lokal, parameter fungsi, dan informasi pemanggilan fungsi. Stack bekerja dengan prinsip Last In First Out, di mana data terakhir yang dimasukkan akan dikeluarkan pertama. Setiap pemanggilan fungsi akan membuat stack frame baru, dan stack frame tersebut akan dihapus secara otomatis

ketika fungsi selesai dieksekusi. Pada arsitektur sistem Linux, stack biasanya tumbuh ke arah alamat memori yang lebih rendah. Pemanggilan fungsi secara rekursif akan membentuk beberapa stack frame, sehingga perubahan alamat stack dapat diamati pada setiap level pemanggilan.

Sistem operasi Linux menerapkan mekanisme keamanan yang disebut Address Space Layout Randomization atau ASLR. ASLR berfungsi untuk mengacak lokasi segmen memori seperti text, data, heap, dan stack setiap kali program dijalankan. Dengan adanya ASLR, alamat memori yang digunakan oleh program akan berbeda pada setiap eksekusi meskipun struktur memory layout tetap sama. Mekanisme ini bertujuan untuk meningkatkan keamanan sistem dengan mencegah serangan yang bergantung pada pengetahuan alamat memori tertentu, seperti buffer overflow dan return-oriented programming.

Linux menyediakan informasi detail mengenai pemetaan memori suatu proses melalui file `/proc/[pid]/maps`. File ini menampilkan rentang alamat memori, jenis segmen, serta hak akses memori yang dinyatakan dalam bentuk read, write, dan execute. Informasi ini dapat digunakan untuk memverifikasi apakah alamat memori yang diperoleh dari hasil pengamatan program sesuai dengan segment memori yang dialokasikan oleh sistem operasi.

Dengan memahami konsep memory layout, pembagian segmen memori, serta mekanisme ASLR, analisis terhadap alamat memori pada program C dapat dilakukan secara sistematis. Hal ini membantu dalam memahami cara kerja manajemen memori, mendeteksi kesalahan pemrograman, serta meningkatkan keamanan dan keandalan aplikasi.

2.2 Contiguous Allocation

Manajemen memori merupakan salah satu fungsi utama sistem operasi yang bertugas mengatur penggunaan memori utama oleh proses yang berjalan. Karena jumlah memori terbatas dan digunakan secara bersamaan oleh banyak proses, sistem operasi harus menyediakan mekanisme alokasi dan dealokasi memori yang efisien agar kinerja sistem tetap optimal dan pemborosan memori dapat diminimalkan.

Pada sistem multiprogramming, memori dibagi menjadi beberapa bagian yang dapat dialokasikan ke proses sesuai kebutuhan. Salah satu pendekatan yang digunakan adalah alokasi memori kontigu, di mana setiap proses memperoleh satu blok memori yang bersebelahan secara fisik. Pendekatan ini sederhana untuk diimplementasikan, namun memiliki kelemahan utama berupa fragmentasi eksternal, yaitu kondisi ketika ruang memori

bebas terpecah menjadi beberapa bagian kecil yang tidak dapat digunakan secara optimal meskipun total ruang bebas masih mencukupi.

Untuk mengatasi permasalahan tersebut, dikembangkan berbagai algoritma alokasi memori.

Algoritma First-Fit mengalokasikan blok bebas pertama yang ukurannya cukup untuk memenuhi permintaan proses. Algoritma ini cepat karena tidak memerlukan pencarian menyeluruh, tetapi cenderung menghasilkan fragmentasi di bagian awal memori. Algoritma

Best-Fit memilih blok bebas dengan ukuran paling mendekati kebutuhan proses, dengan tujuan mengurangi sisa ruang memori yang terbuang. Namun, pendekatan ini dapat

menghasilkan banyak fragmen kecil dan membutuhkan waktu pencarian yang lebih lama.

Algoritma Worst-Fit mengalokasikan blok bebas terbesar yang tersedia dengan harapan sisa ruang masih cukup besar untuk alokasi berikutnya, meskipun dalam praktiknya sering menghasilkan fragmentasi yang tinggi.

Algoritma Next-Fit merupakan pengembangan dari First-Fit. Perbedaannya terletak pada titik awal pencarian blok bebas. Next-Fit tidak selalu memulai pencarian dari awal memori, melainkan dari posisi alokasi terakhir yang berhasil. Pendekatan ini bertujuan untuk mengurangi waktu pencarian berulang pada area memori yang sama serta menyebarkan alokasi secara lebih merata di seluruh memori.

Fragmentasi memori menjadi indikator penting dalam mengevaluasi kinerja algoritma alokasi. Fragmentasi eksternal terjadi ketika memori bebas terbagi menjadi beberapa lubang yang terpisah sehingga tidak dapat digunakan secara optimal. Tingkat fragmentasi dapat dianalisis dengan menghitung total ruang bebas, ukuran blok bebas terbesar, serta jumlah lubang memori yang terbentuk. Semakin kecil blok bebas terbesar dibandingkan total ruang bebas, semakin tinggi tingkat fragmentasi eksternal yang terjadi.

Untuk membandingkan efektivitas masing-masing algoritma, diperlukan pengujian performa menggunakan skenario alokasi dan dealokasi yang sama. Pengukuran waktu eksekusi memberikan gambaran efisiensi algoritma dalam menemukan blok memori yang sesuai, sedangkan perhitungan fragmentasi menunjukkan kualitas penggunaan memori. Dengan analisis tersebut, dapat diketahui kelebihan dan kekurangan masing-masing algoritma dalam mengelola memori utama.

2.3 Paging

Manajemen memori merupakan salah satu komponen utama dalam sistem operasi yang bertugas mengatur pemakaian memori utama agar dapat digunakan secara efisien oleh berbagai proses. Salah satu teknik manajemen memori yang umum digunakan adalah paging. Paging membagi memori fisik menjadi unit berukuran tetap yang disebut frame dan memori logis menjadi unit yang disebut page. Dengan mekanisme ini, proses tidak perlu disimpan secara kontigu di memori fisik, sehingga dapat mengurangi fragmentasi eksternal.

Dalam sistem paging, ketika sebuah proses mengakses halaman yang belum berada di memori utama, akan terjadi page fault. Sistem operasi harus memuat halaman tersebut dari media penyimpanan sekunder ke dalam memori. Karena jumlah frame terbatas, sistem perlu menentukan halaman mana yang harus dikeluarkan ketika memori penuh. Keputusan ini dilakukan oleh algoritma page replacement.

Algoritma FIFO atau First-In First-Out merupakan algoritma page replacement paling sederhana. Algoritma ini mengganti halaman berdasarkan urutan waktu kedatangan ke memori. Halaman yang pertama kali dimuat akan menjadi halaman pertama yang digantikan ketika terjadi page fault dan memori telah penuh. FIFO mudah diimplementasikan dan memiliki overhead rendah, namun tidak mempertimbangkan frekuensi atau pola akses halaman. Akibatnya, FIFO dapat mengganti halaman yang masih sering digunakan, sehingga performanya tidak selalu optimal.

Algoritma LRU atau Least Recently Used mengganti halaman yang paling lama tidak diakses. Algoritma ini didasarkan pada prinsip locality of reference, yaitu kecenderungan program untuk mengakses data dan instruksi yang sama atau berdekatan dalam rentang waktu tertentu. Dengan mempertahankan halaman yang baru saja diakses di memori, LRU mampu menurunkan jumlah page fault dibandingkan FIFO. Namun, LRU membutuhkan mekanisme pencatatan waktu atau urutan akses halaman, sehingga implementasinya lebih kompleks dan memiliki overhead lebih besar.

Performa algoritma page replacement umumnya diukur menggunakan beberapa metrik, antara lain jumlah page fault, page hit, dan hit ratio. Page fault menunjukkan seberapa sering sistem harus mengambil halaman dari disk, sedangkan hit ratio menggambarkan tingkat keberhasilan sistem dalam menyediakan halaman yang dibutuhkan langsung dari memori. Semakin tinggi hit ratio, semakin baik performa sistem.

Dalam beberapa algoritma page replacement, khususnya FIFO, dapat terjadi fenomena yang disebut Belady's Anomaly. Fenomena ini terjadi ketika penambahan jumlah frame justru menyebabkan peningkatan jumlah page fault. Hal ini bertentangan dengan asumsi umum bahwa lebih banyak memori selalu menghasilkan performa yang lebih baik. Algoritma seperti LRU tidak mengalami Belady's Anomaly karena termasuk dalam kategori stack algorithm, di mana kumpulan halaman pada memori dengan n frame selalu merupakan subset dari memori dengan $n+1$ frame.

2.4 Page Table Structures

Manajemen memori merupakan komponen fundamental dalam sistem operasi yang bertanggung jawab mengatur penggunaan memori utama secara efisien dan aman oleh berbagai proses. Salah satu teknik manajemen memori yang paling umum digunakan adalah paging. Pada mekanisme paging, ruang alamat virtual dibagi menjadi unit berukuran tetap yang disebut page, sedangkan memori fisik dibagi menjadi frame dengan ukuran yang sama. Translasi alamat virtual ke alamat fisik dilakukan menggunakan struktur data yang disebut page table.

Single-level page table memetakan setiap halaman virtual ke frame fisik secara langsung. Pendekatan ini sederhana dan cepat, namun menjadi tidak efisien ketika digunakan pada sistem dengan ruang alamat virtual yang besar, karena seluruh page table harus dialokasikan di memori walaupun sebagian besar halaman tidak pernah digunakan. Masalah ini mendorong pengembangan struktur page table bertingkat atau multi-level page table.

Multi-level page table membagi page table menjadi beberapa tingkat hierarkis. Alamat virtual dipecah menjadi beberapa bagian yang masing-masing digunakan sebagai indeks ke page table pada level tertentu. Tabel pada level yang lebih rendah hanya dialokasikan ketika diperlukan, sehingga penggunaan memori menjadi lebih hemat. Semakin banyak level yang digunakan, semakin kecil ukuran tabel yang perlu dialokasikan pada setiap level, namun jumlah akses memori untuk translasi alamat juga meningkat.

Two-level page table umum digunakan pada sistem 32-bit, sedangkan three-level atau bahkan four-level page table banyak diterapkan pada arsitektur 64-bit modern. Struktur ini memungkinkan sistem untuk menangani ruang alamat virtual yang sangat besar tanpa membebani memori utama dengan page table berukuran besar.

Sebagai alternatif dari multi-level page table, inverted page table dikembangkan untuk lebih menghemat penggunaan memori. Inverted page table hanya memiliki satu tabel global dengan jumlah entri yang sama dengan jumlah frame fisik. Setiap entri menyimpan informasi mengenai halaman virtual dan proses yang menggunakan frame tersebut. Dengan pendekatan ini, ukuran page table tidak bergantung pada ukuran ruang alamat virtual, melainkan pada ukuran memori fisik.

Kelemahan inverted page table terletak pada proses translasi alamat yang lebih kompleks. Karena tabel tidak diindeks langsung oleh nomor halaman virtual, sistem harus melakukan pencarian terhadap pasangan process ID dan page number. Untuk mengatasi masalah ini, inverted page table biasanya dikombinasikan dengan teknik hashing dan Translation Lookaside Buffer (TLB) agar translasi alamat tetap efisien.

Translation Lookaside Buffer merupakan cache berkecepatan tinggi yang menyimpan sebagian kecil entri page table yang sering digunakan. Dengan adanya TLB, sebagian besar translasi alamat dapat dilakukan tanpa mengakses page table di memori utama, sehingga mengurangi jumlah akses memori dan meningkatkan performa sistem secara signifikan. Peran TLB menjadi semakin penting pada sistem dengan multi-level page table karena mampu mengurangi overhead akibat banyaknya tahapan translasi.

Berdasarkan teori tersebut, dapat disimpulkan bahwa pemilihan struktur page table merupakan kompromi antara efisiensi penggunaan memori, kecepatan translasi alamat, dan kompleksitas implementasi. Multi-level page table lebih cocok untuk sistem dengan ruang alamat virtual besar, sedangkan inverted page table lebih menekankan efisiensi memori dengan dukungan mekanisme percepatan seperti TLB.

2.5 Swapping

Swapping merupakan salah satu teknik manajemen memori pada sistem operasi yang digunakan untuk mengatasi keterbatasan kapasitas memori fisik. Pada mekanisme ini, proses yang sedang tidak atau jarang digunakan dapat dipindahkan sementara dari memori utama ke media penyimpanan sekunder, seperti disk, sehingga ruang memori dapat digunakan oleh proses lain yang lebih membutuhkan. Ketika proses tersebut kembali diperlukan, sistem operasi akan memuatnya kembali ke memori utama melalui mekanisme swap in. Pendekatan ini memungkinkan sistem untuk menjalankan lebih banyak proses dibandingkan kapasitas memori fisik yang tersedia.

Keputusan mengenai proses mana yang akan di-swap out sangat berpengaruh terhadap kinerja sistem. Oleh karena itu, sistem operasi menerapkan berbagai strategi pemilihan victim untuk meminimalkan overhead swapping dan menghindari thrashing. Thrashing adalah kondisi di mana sistem lebih banyak menghabiskan waktu untuk melakukan swap in dan swap out dibandingkan menjalankan proses secara efektif. Kondisi ini menyebabkan penurunan performa secara drastis dan harus dihindari dalam desain sistem operasi.

Strategi FIFO (First-In First-Out) merupakan pendekatan swapping paling sederhana, di mana proses yang pertama kali dimuat ke memori akan menjadi kandidat pertama untuk dikeluarkan. Strategi ini tidak mempertimbangkan frekuensi maupun pola akses proses, sehingga proses yang masih aktif dapat dikeluarkan hanya karena berada lebih lama di memori. Akibatnya, FIFO rentan menyebabkan thrashing terutama pada beban kerja dengan pola akses dinamis.

Strategi LRU (Least Recently Used) didasarkan pada prinsip locality of reference, yaitu kecenderungan program untuk mengakses kembali data atau instruksi yang baru saja digunakan. Dengan memilih proses yang paling lama tidak diakses sebagai victim, LRU berusaha mempertahankan proses aktif di memori utama. Strategi ini secara teoritis lebih optimal dalam mengurangi page fault dan swapping yang tidak perlu, meskipun membutuhkan overhead tambahan untuk mencatat waktu atau urutan akses proses.

2.6 Architecture Analysis

Arsitektur prosesor menentukan bagaimana sistem operasi mengelola alamat memori, termasuk ukuran address space, mekanisme translasi alamat, dan struktur page table yang digunakan. Address space merupakan rentang alamat virtual yang dapat direpresentasikan oleh prosesor dan digunakan oleh sistem operasi untuk memetakan alamat virtual ke alamat fisik. Semakin besar jumlah bit address space, semakin besar pula ruang alamat virtual yang dapat dikelola oleh sistem operasi.

Pada arsitektur 32-bit, address space dibatasi oleh lebar register alamat sebesar 32 bit, sehingga maksimum alamat yang dapat diakses adalah 2^{32} byte atau 4 GB. Keterbatasan ini berdampak langsung pada desain sistem operasi, terutama dalam hal pembagian ruang alamat antara kernel dan proses pengguna serta ukuran maksimum memori virtual per proses. Untuk mengelola memori secara efisien, arsitektur 32-bit menerapkan paging bertingkat guna mengurangi overhead penyimpanan page table, meskipun ruang alamatnya relatif kecil.

Arsitektur 64-bit memperluas address space secara signifikan dengan menggunakan register alamat berukuran 64 bit. Walaupun secara teoritis mampu mengakses 2^{64} byte memori, implementasi praktis biasanya membatasi jumlah bit alamat virtual yang digunakan, misalnya 48 bit pada banyak sistem operasi modern. Pembatasan ini dilakukan untuk menjaga kompatibilitas perangkat keras dan efisiensi implementasi. Dengan address space yang jauh lebih besar, sistem operasi 64-bit dapat mendukung aplikasi dengan kebutuhan memori besar serta mengurangi tekanan terhadap mekanisme swapping.

Untuk mengelola address space yang besar, arsitektur 64-bit menggunakan multi-level paging. Multi-level paging membagi alamat virtual menjadi beberapa bagian yang masing-masing berfungsi sebagai indeks pada level page table tertentu. Pendekatan ini memungkinkan sistem hanya mengalokasikan page table untuk bagian alamat virtual yang benar-benar digunakan, sehingga menghemat penggunaan memori. Konsep ini berlaku baik pada arsitektur x86-64 maupun ARMv8, meskipun jumlah level dan detail implementasinya dapat berbeda.

Arsitektur ARMv8, sebagai arsitektur 64-bit modern, menerapkan prinsip yang serupa dengan x86-64 dalam pengelolaan address space dan paging. ARMv8 mendukung address space besar dan multi-level page table, namun dirancang dengan fokus pada efisiensi daya dan fleksibilitas, sehingga banyak digunakan pada perangkat mobile hingga server. Dengan dukungan paging bertingkat, ARMv8 mampu menjalankan sistem operasi modern dengan performa yang sebanding dengan arsitektur x86-64.

Secara teoritis, semakin besar address space dan semakin efisien struktur paging yang digunakan, semakin baik sistem operasi dalam mengelola memori virtual. Hal ini memungkinkan isolasi proses yang lebih kuat, dukungan aplikasi berskala besar, serta pengurangan frekuensi swapping. Oleh karena itu, peralihan dari arsitektur 32-bit ke 64-bit merupakan langkah penting dalam evolusi sistem operasi modern.

BAB III

JAWABAN TUGAS PRAKTIKUM

3.1 Analaisis Memory Layout

1. Deskripsi Tugas

Modifikasi program address_demo.c untuk menganalisis memory layout dengan lebih detail.

Yang Harus Dikerjakan

A.Modifikasi Program

Tambahkan fitur-fitur berikut pada program:

- . Array Allocation: Buat array statis dan dinamis, tampilkan alamatnya
- . Multiple Functions: Buat 3 fungsi berbeda, tampilkan alamat setiap fungsi
- . Recursive Function: Buat fungsi rekursif dan tampilkan alamat stack di setiap level
- . String Literals: Tampilkan alamat string literals

Contoh Output yang Diharapkan:

=== ENHANCED MEMORY LAYOUT ===

1. Text Segment:

main() address: 0x...

func1() address: 0x...

func2() address: 0x...

2.Data Segment:

static_array[0] address: 0x... string_literal address: 0x...

3.Heap:

dynamic_array[0] address: 0x... malloc block 1: 0x...

malloc block 2: 0x...

4.Stack (Recursive Calls): Level 0 local_var: 0x... Level 1 local_var: 0x... Level 2 local_var: 0x...

2. Source Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
/* ----- Global & Static (Data Segment) ----- */
```

```

int global_var = 100;
static int static_array[5] = {1, 2, 3, 4, 5};

/* ----- Function Declarations (Text Segment) ----- */
void func1();
void func2();
void recursive_func(int level, int max);

/* ----- Functions ----- */
void func1() {
    int local_func1 = 10;
    printf("func1 local var address : %p\n", (void *)&local_func1);
}

void func2() {
    int local_func2 = 20;
    printf("func2 local var address : %p\n", (void *)&local_func2);
}

void recursive_func(int level, int max) {
    int local_recursive = level;
    printf("Level %d local_var address: %p\n",
        level, (void *)&local_recursive);

    if (level < max) {
        recursive_func(level + 1, max);
    }
}

int main() {
    printf("=== ENHANCED MEMORY LAYOUT ===\n\n");
}

```

```

/* ----- TEXT SEGMENT ----- */
printf("1. TEXT SEGMENT:\n");
printf("main() address : %p\n", (void *)&main);
printf("func1() address: %p\n", (void *)&func1);
printf("func2() address: %p\n\n", (void *)&func2);

/* ----- DATA SEGMENT ----- */
const char *string_literal = "Hello Memory Layout";

printf("2. DATA SEGMENT:\n");
printf("global_var address    : %p\n", (void *)&global_var);
printf("static_array[0] address : %p\n", (void *)&static_array[0]);
printf("string_literal address  : %p\n\n", (void *)string_literal);

/* ----- HEAP ----- */
printf("3. HEAP:\n");
int *dynamic_array = (int *)malloc(5 * sizeof(int));
int *block1 = (int *)malloc(10 * sizeof(int));
int *block2 = (int *)malloc(20 * sizeof(int));

printf("dynamic_array[0] address: %p\n", (void *)&dynamic_array[0]);
printf("malloc block 1 address : %p\n", (void *)block1);
printf("malloc block 2 address : %p\n\n", (void *)block2);

/* ----- STACK & RECURSION ----- */
printf("4. STACK (RECURSIVE CALLS):\n");
recursive_func(0, 2);
printf("\n");

/* ----- PID FOR /proc/maps ----- */
printf("Process PID: %d\n", getpid());
printf("Check memory map with:\n");

```



```

printf("cat /proc/%d/maps\n", getpid());

/* ----- CLEANUP ----- */
free(dynamic_array);
free(block1);
free(block2);

return 0;
}

```

3. Screenshot Output

```

=== ENHANCED MEMORY LAYOUT ===

1. TEXT SEGMENT:
main() address : 0x57e0b7295326
func1() address: 0x57e0b7295209
func2() address: 0x57e0b729525d

2. DATA SEGMENT:
global_var address      : 0x57e0b7298010
static_array[0] address : 0x57e0b7298020
string_literal address  : 0x57e0b72960d9

3. HEAP:
dynamic_array[0] address: 0x57e0e2e426b0
malloc block 1 address  : 0x57e0e2e426d0
malloc block 2 address  : 0x57e0e2e42700

4. STACK (RECURSIVE CALLS):
Level 0 local_var address: 0x7ffdd2453744
Level 1 local_var address: 0x7ffdd2453714
Level 2 local_var address: 0x7ffdd24536e4

Process PID: 1222
Check memory map with:
cat /proc/1222/maps

```

4. Penjelasan Implementasi

Program ini dibuat untuk mengamati tata letak memori (memory layout) pada program C, meliputi:

- a. Text segment
- b. Data segment
- c. Heap

d. Stack

serta melihat pengaruh ASLR (Address Space Layout Randomization) pada alamat memori.

Program terdiri dari:

- Variabel global dan statis
- Beberapa fungsi
- Alokasi memori statis dan dinamis
- Fungsi rekursif
- String literal
- Pengambilan PID untuk pengecekan `/proc/[pid]/maps`

A. Text Segment (Kode Program)

Implementasi

```
printf("main() address : %p\n", (void *)&main);  
printf("func1() address: %p\n", (void *)&func1);  
printf("func2() address: %p\n", (void *)&func2);
```

- `main`, `func1`, dan `func2` adalah **fungsi**
- Alamat fungsi berada di **text segment**
- Text segment:

Bersifat **read-only**

Memiliki permission `r-x`

- dengan mencetak alamat fungsi, kita dapat:

Melihat letak text segment

Mengukur jarak antar fungsi

B. Data Segment (Global, Static, dan String Literal)

Implementasi

```
int global_var = 100;  
static int static_array[5] = {1, 2, 3, 4, 5};  
const char *string_literal = "Hello Memory Layout";
```

```
printf("global_var address : %p\n", (void *)&global_var);  
printf("static_array[0] address : %p\n", (void *)&static_array[0]);  
printf("string_literal address : %p\n", (void *)string_literal);
```

- `global_var` : Disimpan di data segment
- `static_array` : Juga berada di data segment
- `string_literal` : Disimpan di read-only data segment dan tidak bisa dimodifikasi
- Data segment : Dialokasikan saat program dimuat dan bersifat persisten selama program berjalan

C. Stack (Variabel Lokal & Fungsi Biasa)

implementasi

```
void func1() {
    int local_func1 = 10;
    printf("func1 local var address : %p\n", (void *)&local_func1);
}
```

- `local_func1` berada di stack
- Stack:
Digunakan untuk menyimpan variabel lokal
Bersifat LIFO (Last In First Out)
Dialokasikan otomatis
- Setiap pemanggilan fungsi: Membuat stack frame baru

3.2 Memory Allocator

3.2.1 Deskripsi Tugas

Kembangkan program memory allocator dengan fitur tambahan dan lakukan analisis performa.

Yang Harus Dikerjakan

A.Implementasi Next-Fit Algorithm

Tambahkan algoritma Next-Fit pada program `memory_allocator.c`:

Next-Fit: Seperti First-Fit, tapi mulai pencarian dari posisi alokasi terakhir

```
int next_fit(int process_id, int size) {
    // TODO: Implementasi Next-Fit
    // Gunakan variabel global untuk menyimpan posisi terakhir
}
```

B.Fitur Fragmentasi

Tambahkan fungsi untuk menghitung fragmentasi:

```
void calculate_fragmentation() {
```

```
// Hitung:
// 1. Total free space
// 2. Largest free block
// 3. Number of free holes
// 4. External fragmentation ratio
}
```

C. Benchmark Testing

Buat test case untuk membandingkan 4 algoritma:

```
void benchmark_test() {
// Test case:
// 1. Allocate: P1(100), P2(200), P3(300), P4(250), P5(150)
// 2. Deallocate: P2, P4
// 3. Allocate: P6(180), P7(220), P8(100)
// 4. Hitung waktu eksekusi dan fragmentasi untuk setiap algoritma
}
```

3.2.2 Source Code

```
#include <stdio.h>
#include <time.h>

#define MEMORY_SIZE 1000
#define MAX_BLOCKS 100

/* ----- Struktur Memory Block ----- */
typedef struct {
    int start;
    int size;
    int free;
    int pid;
} Block;

Block memory[MAX_BLOCKS];
```

```

int block_count;

/* ----- Next-Fit Pointer ----- */
int last_alloc_pos = 0;

/* ----- Inisialisasi Memory ----- */
void init_memory() {
    block_count = 1;
    memory[0].start = 0;
    memory[0].size = MEMORY_SIZE;
    memory[0].free = 1;
    memory[0].pid = -1;
    last_alloc_pos = 0;
}

/* ----- Split Block ----- */
void split_block(int index, int pid, int size) {
    Block new_block;
    new_block.start = memory[index].start + size;
    new_block.size = memory[index].size - size;
    new_block.free = 1;
    new_block.pid = -1;

    memory[index].size = size;
    memory[index].free = 0;
    memory[index].pid = pid;

    for (int i = block_count; i > index + 1; i--) {
        memory[i] = memory[i - 1];
    }
    memory[index + 1] = new_block;
    block_count++;
}

```

```

}

/* ----- First-Fit ----- */
int first_fit(int pid, int size) {
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            split_block(i, pid, size);
            return 1;
        }
    }
    return 0;
}

/* ----- Best-Fit ----- */
int best_fit(int pid, int size) {
    int best = -1;
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (best == -1 || memory[i].size < memory[best].size) {
                best = i;
            }
        }
    }
    if (best != -1) {
        split_block(best, pid, size);
        return 1;
    }
    return 0;
}

/* ----- Worst-Fit ----- */
int worst_fit(int pid, int size) {

```

```

int worst = -1;
for (int i = 0; i < block_count; i++) {
    if (memory[i].free && memory[i].size >= size) {
        if (worst == -1 || memory[i].size > memory[worst].size) {
            worst = i;
        }
    }
}
if (worst != -1) {
    split_block(worst, pid, size);
    return 1;
}
return 0;
}

```

/* ----- Next-Fit ----- */

```

int next_fit(int pid, int size) {
    int i = last_alloc_pos;
    int scanned = 0;

    while (scanned < block_count) {
        if (memory[i].free && memory[i].size >= size) {
            split_block(i, pid, size);
            last_alloc_pos = i;
            return 1;
        }
        i = (i + 1) % block_count;
        scanned++;
    }
    return 0;
}

```

```

/* ----- Deallocate ----- */
void deallocate(int pid) {
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].free && memory[i].pid == pid) {
            memory[i].free = 1;
            memory[i].pid = -1;
        }
    }
}

/* ----- Fragmentation ----- */
void calculate_fragmentation() {
    int total_free = 0;
    int largest_free = 0;
    int holes = 0;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free) {
            total_free += memory[i].size;
            holes++;
            if (memory[i].size > largest_free) {
                largest_free = memory[i].size;
            }
        }
    }

    double ext_frag = 0.0;
    if (total_free > 0) {
        ext_frag = 1.0 - ((double)largest_free / total_free);
    }

    printf("Total Free Space      : %d\n", total_free);
}

```



```

printf("Largest Free Block    : %d\n", largest_free);
printf("Number of Free Holes   : %d\n", holes);
printf("External Fragmentation : %.2f\n", ext_frag);
}

/* ----- Benchmark ----- */
void benchmark_test(int (*alloc)(int, int), const char *name) {
    init_memory();
    clock_t start = clock();

    alloc(1,100);
    alloc(2,200);
    alloc(3,300);
    alloc(4,250);
    alloc(5,150);

    deallocate(2);
    deallocate(4);

    alloc(6,180);
    alloc(7,220);
    alloc(8,100);

    clock_t end = clock();
    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("\n=== %s ===\n", name);
    printf("Execution Time: %f seconds\n", time_spent);
    calculate_fragmentation();
}

/* ----- MAIN ----- */

```

```

int main() {
    benchmark_test(first_fit, "First-Fit");
    benchmark_test(best_fit, "Best-Fit");
    benchmark_test(worst_fit, "Worst-Fit");
    benchmark_test(next_fit, "Next-Fit");
    return 0;
}

```

3.2.3 Screenshot Output

```

=== First-Fit ===
Execution Time: 0.000004 seconds
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 3
External Fragmentation : 0.40

=== Best-Fit ===
Execution Time: 0.000003 seconds
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 3
External Fragmentation : 0.40

=== Worst-Fit ===
Execution Time: 0.000024 seconds
Total Free Space      : 170
Largest Free Block    : 100
Number of Free Holes  : 3
External Fragmentation : 0.41

=== Next-Fit ===
Execution Time: 0.000002 seconds
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 3
External Fragmentation : 0.40

```

3.2.4 Penjelasan Implementasi

Program memory allocator ini dirancang untuk mensimulasikan pengelolaan memori utama dengan menggunakan beberapa algoritma alokasi, yaitu First-Fit, Best-Fit, Worst-Fit, dan Next-Fit. Seluruh algoritma bekerja pada sebuah array yang merepresentasikan ruang memori berukuran tetap. Setiap bagian memori direpresentasikan sebagai sebuah blok yang memiliki informasi posisi awal, ukuran, status bebas atau terpakai, serta identitas proses yang menggunakan blok tersebut.

Pada saat program dijalankan, memori diinisialisasi sebagai satu blok besar yang seluruhnya berstatus bebas. Proses alokasi dilakukan dengan membagi blok bebas menjadi dua bagian apabila ukuran blok lebih besar dari kebutuhan proses. Satu bagian digunakan oleh proses

yang meminta memori, sedangkan sisa ruang tetap disimpan sebagai blok bebas. Mekanisme pemisahan blok ini memungkinkan simulasi penggunaan memori yang lebih realistis dan memunculkan fragmentasi eksternal.

Algoritma First-Fit diimplementasikan dengan cara melakukan pencarian blok bebas dari awal memori hingga menemukan blok pertama yang cukup besar untuk memenuhi permintaan alokasi. Algoritma ini sederhana dan memiliki waktu pencarian yang relatif cepat, namun cenderung menghasilkan fragmentasi di bagian awal memori karena selalu menggunakan blok pertama yang tersedia.

Algoritma Best-Fit bekerja dengan memilih blok bebas yang ukurannya paling mendekati kebutuhan proses. Implementasi dilakukan dengan melakukan pencarian seluruh blok bebas dan menentukan blok dengan sisa ruang terkecil setelah alokasi. Pendekatan ini bertujuan meminimalkan sisa memori yang terbuang, tetapi dapat meningkatkan fragmentasi kecil yang tersebar dan membutuhkan waktu pencarian lebih lama karena harus memeriksa seluruh blok. Algoritma Worst-Fit memilih blok bebas dengan ukuran terbesar untuk dialokasikan ke proses. Implementasinya mirip dengan Best-Fit, tetapi blok yang dipilih adalah blok dengan ukuran paling besar. Tujuan pendekatan ini adalah mempertahankan blok bebas berukuran besar agar dapat digunakan oleh proses berikutnya, meskipun pada praktiknya algoritma ini sering menghasilkan fragmentasi yang cukup tinggi.

Algoritma Next-Fit dikembangkan sebagai variasi dari First-Fit. Perbedaannya terletak pada titik awal pencarian. Pencarian blok bebas tidak selalu dimulai dari awal memori, melainkan dari posisi terakhir alokasi yang berhasil dilakukan. Posisi tersebut disimpan dalam variabel global dan digunakan sebagai titik awal pencarian berikutnya. Implementasi ini bertujuan untuk mengurangi waktu pencarian pada area memori yang sama secara berulang dan menyebarkan alokasi secara lebih merata di seluruh memori.

Fitur perhitungan fragmentasi ditambahkan untuk mengevaluasi efisiensi penggunaan memori oleh masing-masing algoritma. Perhitungan dilakukan dengan menjumlahkan seluruh ruang memori yang masih bebas, menentukan ukuran blok bebas terbesar, serta menghitung jumlah lubang memori yang terbentuk. Rasio fragmentasi eksternal dihitung dengan membandingkan ukuran blok bebas terbesar terhadap total ruang bebas, sehingga dapat menggambarkan seberapa terpecah ruang memori yang tersedia.

Pengujian performa dilakukan melalui fungsi benchmark yang menjalankan skenario alokasi dan dealokasi yang sama pada setiap algoritma. Skenario ini mencakup beberapa proses yang

dialokasikan secara berurutan, diikuti oleh dealokasi beberapa proses di tengah, kemudian dilanjutkan dengan alokasi proses baru. Waktu eksekusi setiap algoritma diukur menggunakan fungsi pengukur waktu, dan hasil fragmentasi dihitung setelah seluruh operasi selesai. Dengan pendekatan ini, perbandingan kinerja dan tingkat fragmentasi antar algoritma dapat dilakukan secara objektif.

Secara keseluruhan, implementasi ini menunjukkan perbedaan karakteristik masing-masing algoritma alokasi memori. First-Fit dan Next-Fit cenderung lebih cepat dalam pencarian, sementara Best-Fit dan Worst-Fit lebih intensif dalam pencarian blok. Pengukuran fragmentasi memberikan gambaran nyata mengenai dampak strategi alokasi terhadap efisiensi penggunaan memori.

3.3 Advanced Paging System

3.3.1 Deskripsi Tugas

Kembangkan sistem paging dengan page replacement algorithm dan statistik.

Yang Harus Dikerjakan

A.Implementasi Page Replacement - FIFO

Modifikasi paging_simulator.c untuk menangani situasi ketika semua frame penuh:

```
int fifo_page_replacement(int new_page) {  
    // TODO: Implementasi FIFO  
    // 1. Cari page yang paling lama di memory  
    // 2. Evict page tersebut  
    // 3. Load page baru  
    // 4. Update page table  
}
```

B.Implementasi Page Replacement - LRU

Tambahkan algoritma LRU (Least Recently Used):

```
int lru_page_replacement(int new_page) {  
    // TODO: Implementasi LRU  
    // 1. Track waktu akses setiap page
```

```
// 2. Cari page yang paling lama tidak diakses
// 3. Evict page tersebut
}
```

C.Statistik dan Analisis

Tambahkan fungsi untuk menghitung:

```
typedef struct {
    int total_accesses; int page_faults; int page_hits; float hit_ratio;
    int disk_writes;      // untuk dirty pages
} PageStatistics;
void display_statistics(PageStatistics *stats) {
    // Tampilkan statistik
}
```

D.Test Scenario

Jalankan reference string berikut dengan 4 frames:

Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0,
1

3.3.2 Source Code

```
#include <stdio.h>
#include <stdbool.h>
```

```
#define FRAMES 4
#define PAGES 50
#define REF_LEN 20
```

```
typedef struct {
    int total_accesses;
    int page_faults;
    int page_hits;
    float hit_ratio;
    int disk_writes;
```

```

} PageStatistics;

/* GLOBAL */
int frames[FRAMES];
int fifo_index = 0;
int time_counter = 0;
int last_used[PAGES];

/* INIT MEMORY */
void init_memory() {
    for (int i = 0; i < FRAMES; i++)
        frames[i] = -1;

    for (int i = 0; i < PAGES; i++)
        last_used[i] = -1;

    fifo_index = 0;
    time_counter = 0;
}

/* CHECK PAGE HIT */
bool page_in_memory(int page) {
    for (int i = 0; i < FRAMES; i++)
        if (frames[i] == page)
            return true;
    return false;
}

/* FIFO PAGE REPLACEMENT */
void fifo_page_replacement(int page) {
    frames[fifo_index] = page;
    fifo_index = (fifo_index + 1) % FRAMES;
}

```

```

}

/* LRU PAGE REPLACEMENT */
void lru_page_replacement(int page) {
    int lru_page = frames[0];
    int lru_index = 0;

    for (int i = 1; i < FRAMES; i++) {
        if (last_used[frames[i]] < last_used[lru_page]) {
            lru_page = frames[i];
            lru_index = i;
        }
    }
    frames[lru_index] = page;
}

/* DISPLAY MEMORY */
void display_frames() {
    for (int i = 0; i < FRAMES; i++) {
        if (frames[i] == -1)
            printf(" - ");
        else
            printf(" %d ", frames[i]);
    }
    printf("\n");
}

/* DISPLAY STATISTICS */
void display_statistics(PageStatistics *stats) {
    stats->hit_ratio = (float)stats->page_hits / stats->total_accesses;

    printf("\nSTATISTICS\n");
}

```

```

printf("Total Accesses : %d\n", stats->total_accesses);
printf("Page Faults   : %d\n", stats->page_faults);
printf("Page Hits     : %d\n", stats->page_hits);
printf("Hit Ratio     : %.2f\n", stats->hit_ratio);
printf("Disk Writes   : %d\n", stats->disk_writes);
}

/* SIMULATION */
void simulate(int reference[], int length, bool use_lru) {
    PageStatistics stats = {0, 0, 0, 0.0, 0};
    init_memory();

    printf(use_lru ? "\n=== LRU SIMULATION ===\n" : "\n=== FIFO SIMULATION\n\n");

    for (int i = 0; i < length; i++) {
        int page = reference[i];
        stats.total_accesses++;

        printf("Access page %d | Frames:", page);

        if (page_in_memory(page)) {
            stats.page_hits++;
        } else {
            stats.page_faults++;

            bool placed = false;
            for (int j = 0; j < FRAMES; j++) {
                if (frames[j] == -1) {
                    frames[j] = page;
                    placed = true;
                    break;
                }
            }
        }
    }
}

```



```

    }
}

if (!placed) {
    if (use_lru)
        lru_page_replacement(page);
    else
        fifo_page_replacement(page);
}
}

last_used[page] = time_counter++;
display_frames();
}

display_statistics(&stats);
}

int main() {
    int reference[REF_LEN] = {
        7, 0, 1, 2, 0, 3, 0, 4, 2, 3,
        0, 3, 2, 1, 2, 0, 1, 7, 0, 1
    };

    simulate(reference, REF_LEN, false); // FIFO
    simulate(reference, REF_LEN, true); // LRU

    return 0;
}

```

3.3.3 Screenshot Output

FIFO simulation

```
=== FIFO SIMULATION ===
Access page 7 | Frames: 7 - - -
Access page 0 | Frames: 7 0 - -
Access page 1 | Frames: 7 0 1 -
Access page 2 | Frames: 7 0 1 2
Access page 0 | Frames: 7 0 1 2
Access page 3 | Frames: 3 0 1 2
Access page 0 | Frames: 3 0 1 2
Access page 4 | Frames: 3 4 1 2
Access page 2 | Frames: 3 4 1 2
Access page 3 | Frames: 3 4 1 2
Access page 0 | Frames: 3 4 0 2
Access page 3 | Frames: 3 4 0 2
Access page 2 | Frames: 3 4 0 2
Access page 1 | Frames: 3 4 0 1
Access page 2 | Frames: 2 4 0 1
Access page 0 | Frames: 2 4 0 1
Access page 1 | Frames: 2 4 0 1
Access page 7 | Frames: 2 7 0 1
Access page 0 | Frames: 2 7 0 1
Access page 1 | Frames: 2 7 0 1

STATISTICS
Total Accesses : 20
Page Faults    : 10
Page Hits      : 10
Hit Ratio      : 0.50
Disk Writes    : 0
```

LRU simulation

```
=== LRU SIMULATION ===
Access page 7 | Frames: 7 - - -
Access page 0 | Frames: 7 0 - -
Access page 1 | Frames: 7 0 1 -
Access page 2 | Frames: 7 0 1 2
Access page 0 | Frames: 7 0 1 2
Access page 3 | Frames: 3 0 1 2
Access page 0 | Frames: 3 0 1 2
Access page 4 | Frames: 3 0 4 2
Access page 2 | Frames: 3 0 4 2
Access page 3 | Frames: 3 0 4 2
Access page 0 | Frames: 3 0 4 2
Access page 3 | Frames: 3 0 4 2
Access page 2 | Frames: 3 0 4 2
Access page 1 | Frames: 3 0 1 2
Access page 2 | Frames: 3 0 1 2
Access page 0 | Frames: 3 0 1 2
Access page 1 | Frames: 3 0 1 2
Access page 7 | Frames: 7 0 1 2
Access page 0 | Frames: 7 0 1 2
Access page 1 | Frames: 7 0 1 2

STATISTICS
Total Accesses : 20
Page Faults    : 8
Page Hits      : 12
Hit Ratio      : 0.60
Disk Writes    : 0
```

3.3.4 Penjelasan Implementasi

Program paging simulator ini dibuat untuk mensimulasikan mekanisme manajemen memori berbasis paging dengan dua algoritma page replacement, yaitu FIFO dan LRU. Sistem menggunakan jumlah frame tetap sebanyak empat buah, dan setiap akses halaman akan dicek apakah halaman tersebut sudah berada di memori atau belum.

Struktur utama yang digunakan adalah array `frames[]` yang merepresentasikan frame fisik di memori utama. Nilai `-1` menandakan frame masih kosong. Selain itu digunakan array `last_used[]` untuk mencatat waktu terakhir sebuah halaman diakses, yang diperlukan oleh algoritma LRU. Variabel `time_counter` berfungsi sebagai penanda waktu global yang terus bertambah setiap kali terjadi akses halaman.

Pada awal simulasi, fungsi `init_memory()` mengosongkan seluruh frame dan menginisialisasi seluruh pencatatan waktu akses. Hal ini memastikan setiap simulasi FIFO dan LRU dimulai dari kondisi memori yang sama agar hasilnya dapat dibandingkan secara adil.

Setiap kali sebuah halaman diakses, program terlebih dahulu memeriksa apakah halaman tersebut sudah berada di memori menggunakan fungsi `page_in_memory()`. Jika halaman ditemukan di dalam frame, maka terjadi page hit dan statistik page hit akan ditambah. Jika halaman tidak ditemukan, maka terjadi page fault dan sistem harus memuat halaman baru ke dalam memori.

Ketika masih terdapat frame kosong, halaman baru langsung dimasukkan ke frame tersebut tanpa melakukan penggantian halaman. Namun, jika seluruh frame telah terisi, maka mekanisme page replacement akan dijalankan sesuai dengan algoritma yang digunakan.

Pada algoritma FIFO, penggantian halaman dilakukan berdasarkan urutan kedatangan halaman ke memori. Program menggunakan variabel `fifo_index` untuk menyimpan posisi frame yang paling lama berada di memori. Setiap terjadi penggantian, halaman baru akan ditempatkan pada posisi `fifo_index`, lalu indeks tersebut digeser secara melingkar. Dengan cara ini, halaman yang pertama kali masuk ke memori akan menjadi halaman pertama yang dikeluarkan.

Pada algoritma LRU, penggantian halaman dilakukan berdasarkan waktu akses terakhir. Program mencari halaman di dalam frame yang memiliki nilai `last_used` paling kecil, yang berarti halaman tersebut paling lama tidak diakses. Frame yang berisi halaman tersebut akan digantikan dengan halaman baru. Setiap kali sebuah halaman diakses, nilai `last_used`

halaman tersebut diperbarui menggunakan `time_counter`, sehingga sistem selalu memiliki informasi terbaru mengenai urutan penggunaan halaman.

Selama simulasi berjalan, program mencatat jumlah total akses halaman, jumlah page fault, dan jumlah page hit ke dalam struktur `PageStatistics`. Setelah seluruh reference string diproses, program menghitung hit ratio sebagai perbandingan antara page hit dan total akses halaman, lalu menampilkan seluruh statistik tersebut ke layar.

Hasil simulasi menunjukkan bahwa algoritma LRU menghasilkan jumlah page fault yang lebih sedikit dibandingkan FIFO karena mempertahankan halaman yang sering digunakan di memori. FIFO lebih sederhana dan cepat dalam implementasi, tetapi tidak memperhatikan pola akses halaman sehingga kurang optimal dalam kondisi nyata.

3.4 Multi-Level Page Table

3.4.1 Deskripsi Tugas

Implementasikan dan bandingkan berbagai struktur page table.

Yang Harus Dikerjakan

A. Three-Level Page Table

Kembangkan three-level page table system:

// Configuration: $16 + 8 + 8 + 12 = 44$ bits (untuk demo)

```
#define L1_BITS 16
```

```
#define L2_BITS 8
```

```
#define L3_BITS 8
```

```
#define OFFSET_BITS 12
```

```
typedef struct {
```

```
    int frame_number; int valid;
```

```
    int read; int write; int execute;
```

```
} L3_Entry;
```

```
typedef struct { L3_Entry *entries; int valid;
```

```
} L2_Entry;
```

```
typedef struct { L2_Entry *entries; int valid;
```

```
} L1_Entry;
```

```
// Implementasi:
```

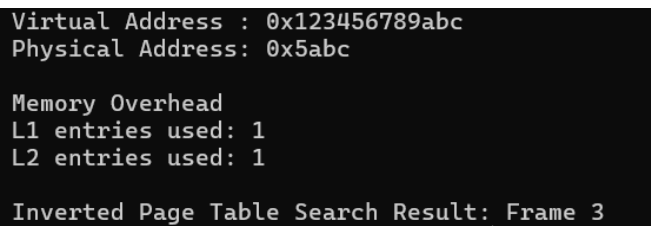
```
int translate_3level(unsigned long virtual_addr);
void allocate_3level(unsigned long virtual_addr, int frame); void
calculate_memory_overhead();
```

B. Inverted Page Table (8 poin)

Implementasikan inverted page table:

```
typedef struct { int process_id;
int page_number; int valid;
} InvertedPageEntry;
InvertedPageEntry inverted_table[NUM_FRAMES]; int search_inverted_table(int pid, int
page_num);
void insert_inverted_table(int pid, int page_num, int frame);
```

3.4.1 Screenshot Output



```
Virtual Address : 0x123456789abc
Physical Address: 0x5abc

Memory Overhead
L1 entries used: 1
L2 entries used: 1

Inverted Page Table Search Result: Frame 3
```

3.4.1 Source Code

```
#include <stdio.h>
#include <stdlib.h>

#define L1_BITS 16
#define L2_BITS 8
#define L3_BITS 8
#define OFFSET_BITS 12

#define NUM_FRAMES 64

typedef struct {
    int frame_number;
```

```

    int valid;
    int read, write, execute;
} L3_Entry;

typedef struct {
    L3_Entry *entries;
    int valid;
} L2_Entry;

typedef struct {
    L2_Entry *entries;
    int valid;
} L1_Entry;

/* ROOT PAGE TABLE */
L1_Entry *page_directory = NULL;

/* THREE LEVEL TRANSLATION */
int translate_3level(unsigned long virtual_addr) {
    unsigned long l1 = (virtual_addr >> (L2_BITS + L3_BITS + OFFSET_BITS)) & ((1 <<
L1_BITS) - 1);
    unsigned long l2 = (virtual_addr >> (L3_BITS + OFFSET_BITS)) & ((1 << L2_BITS) -
1);
    unsigned long l3 = (virtual_addr >> OFFSET_BITS) & ((1 << L3_BITS) - 1);
    unsigned long offset = virtual_addr & ((1 << OFFSET_BITS) - 1);

    if (!page_directory[l1].valid) return -1;
    if (!page_directory[l1].entries[l2].valid) return -1;
    if (!page_directory[l1].entries[l2].entries[l3].valid) return -1;

    int frame = page_directory[l1].entries[l2].entries[l3].frame_number;
    return (frame << OFFSET_BITS) | offset;
}

```

```

}

void allocate_3level(unsigned long virtual_addr, int frame) {
    unsigned long l1 = (virtual_addr >> (L2_BITS + L3_BITS + OFFSET_BITS)) & ((1 <<
L1_BITS) - 1);
    unsigned long l2 = (virtual_addr >> (L3_BITS + OFFSET_BITS)) & ((1 << L2_BITS) -
1);
    unsigned long l3 = (virtual_addr >> OFFSET_BITS) & ((1 << L3_BITS) - 1);

    if (!page_directory[l1].valid) {
        page_directory[l1].entries = calloc(1 << L2_BITS, sizeof(L2_Entry));
        page_directory[l1].valid = 1;
    }

    if (!page_directory[l1].entries[l2].valid) {
        page_directory[l1].entries[l2].entries = calloc(1 << L3_BITS, sizeof(L3_Entry));
        page_directory[l1].entries[l2].valid = 1;
    }

    L3_Entry *entry = &page_directory[l1].entries[l2].entries[l3];
    entry->frame_number = frame;
    entry->valid = 1;
    entry->read = entry->write = entry->execute = 1;
}

void calculate_memory_overhead() {
    int l1_used = 0, l2_used = 0, l3_used = 0;

    for (int i = 0; i < (1 << L1_BITS); i++) {
        if (page_directory[i].valid) {
            l1_used++;
            for (int j = 0; j < (1 << L2_BITS); j++) {

```

```

        if (page_directory[i].entries[j].valid)
            l2_used++;
    }
}

printf("\nMemory Overhead\n");
printf("L1 entries used: %d\n", l1_used);
printf("L2 entries used: %d\n", l2_used);
}

/* INVERTED PAGE TABLE */
typedef struct {
    int process_id;
    int page_number;
    int valid;
} InvertedPageEntry;

InvertedPageEntry inverted_table[NUM_FRAMES];

int search_inverted_table(int pid, int page_num) {
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (inverted_table[i].valid &&
            inverted_table[i].process_id == pid &&
            inverted_table[i].page_number == page_num)
            return i;
    }
    return -1;
}

void insert_inverted_table(int pid, int page_num, int frame) {
    inverted_table[frame].process_id = pid;

```



```

    inverted_table[frame].page_number = page_num;
    inverted_table[frame].valid = 1;
}

int main() {
    page_directory = calloc(1 << L1_BITS, sizeof(L1_Entry));

    unsigned long vaddr = 0x123456789ABC;
    allocate_3level(vaddr, 5);

    int phys = translate_3level(vaddr);
    printf("Virtual Address : 0x%lx\n", vaddr);
    printf("Physical Address: 0x%x\n", phys);

    calculate_memory_overhead();

    insert_inverted_table(1, 10, 3);
    int frame = search_inverted_table(1, 10);
    printf("\nInverted Page Table Search Result: Frame %d\n", frame);

    return 0;
}

```

3.4.1 Penjelasan Implementasi

Mensimulasikan mekanisme translasi alamat virtual ke alamat fisik dengan membagi alamat virtual menjadi empat bagian, yaitu indeks level pertama, indeks level kedua, indeks level ketiga, dan offset. Pembagian ini dilakukan menggunakan operasi bit shifting dan masking sesuai dengan konfigurasi jumlah bit pada masing-masing level. Pendekatan ini meniru cara kerja sistem operasi dalam memetakan alamat virtual berukuran besar secara bertahap melalui beberapa tingkat page table.

Struktur page table level pertama direpresentasikan oleh array `page_directory` yang berisi entri level kedua. Setiap entri pada level pertama dan level kedua memiliki penanda valid untuk menunjukkan apakah tabel pada level tersebut sudah dialokasikan atau belum. Tabel

level bawah hanya dialokasikan ketika terjadi permintaan pemetaan alamat virtual pada indeks tersebut. Mekanisme ini menerapkan prinsip demand paging sehingga penggunaan memori lebih efisien dibandingkan alokasi penuh sejak awal.

Ketika fungsi `allocate_3level` dipanggil, program menghitung indeks level pertama, kedua, dan ketiga dari alamat virtual yang diberikan. Jika entri pada level pertama atau kedua belum valid, program akan mengalokasikan memori untuk tabel level berikutnya menggunakan fungsi `calloc`. Setelah tabel level ketiga tersedia, entri pada level tersebut diisi dengan nomor frame fisik serta bit valid dan bit proteksi yang meliputi izin baca, tulis, dan eksekusi.

Fungsi `translate_3level` bertugas melakukan translasi alamat virtual ke alamat fisik. Proses translasi dilakukan dengan menelusuri page table secara berurutan mulai dari level pertama hingga level ketiga. Jika pada salah satu level entri tidak valid, fungsi akan mengembalikan nilai error yang menandakan terjadinya page fault. Jika seluruh level valid, nomor frame fisik yang ditemukan akan digabungkan dengan offset untuk menghasilkan alamat fisik akhir.

Untuk analisis penggunaan memori, fungsi `calculate_memory_overhead` menghitung jumlah entri page table yang benar-benar digunakan pada level pertama dan kedua. Informasi ini digunakan untuk menunjukkan bahwa three-level page table hanya menggunakan sebagian kecil dari keseluruhan struktur page table yang mungkin, sehingga lebih hemat memori dibandingkan single-level page table.

Implementasi inverted page table dilakukan menggunakan satu array global yang ukurannya tetap berdasarkan jumlah frame fisik. Setiap entri inverted page table menyimpan informasi process ID, nomor halaman virtual, dan status valid. Pendekatan ini berbeda dengan three-level page table karena translasi tidak dilakukan berdasarkan alamat virtual secara langsung, melainkan melalui pencarian pasangan process ID dan nomor halaman pada tabel global.

Fungsi `insert_inverted_table` digunakan untuk mencatat pemetaan halaman virtual ke frame fisik dengan menempatkan informasi tersebut pada indeks frame yang sesuai. Fungsi `search_inverted_table` melakukan pencarian secara linear terhadap inverted page table untuk menemukan frame yang memuat halaman virtual milik suatu proses. Jika pasangan process ID dan nomor halaman ditemukan, fungsi mengembalikan nomor frame, dan jika tidak ditemukan maka dianggap terjadi page fault.

Secara keseluruhan, implementasi ini menunjukkan perbedaan mendasar antara three-level page table dan inverted page table. Three-level page table menawarkan translasi yang cepat dengan struktur hierarkis namun membutuhkan beberapa level akses memori, sedangkan inverted page table menghemat penggunaan memori page table tetapi membutuhkan proses pencarian tambahan yang dapat memengaruhi performa. Perbandingan ini menggambarkan trade-off yang umum terjadi dalam desain manajemen memori pada sistem operasi.

3.5 Swapping Mechanism

3.5.1 Deskripsi Tugas

Simulasikan mekanisme swapping dengan berbagai strategi.

Yang Harus Dikerjakan

A. Basic Swapping Implementation

Buat program untuk mensimulasikan swap in/out:

```
typedef struct { int pid;  
int size;  
int priority; int in_memory; int swap_count;  
time_t last_access;  
} Process;  
void swap_out(Process *p); void swap_in(Process *p);  
void select_victim(); // Pilih proses untuk di-swap out
```

B. Swap Strategy Implementation Implementasikan 3 strategi victim selection:

- . FIFO: Swap out proses yang paling lama di memory
- . LRU: Swap out proses yang paling lama tidak diakses
- . Priority-based: Swap out proses dengan prioritas rendah

C. Simulation Scenario Test dengan skenario:

Memory fisik: 1024 MB

10 proses dengan ukuran berbeda (50-200 MB) Total kebutuhan memory: 1500 MB

Simulasi akses random selama 100 time units

3.5.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_PROCESS 10
#define MEMORY_SIZE 1024 // MB
#define SIM_TIME 100

typedef struct {
    int pid;
    int size;
    int priority;
    int in_memory;
    int swap_count;
    time_t last_access;
    time_t load_time;
} Process;

Process processes[MAX_PROCESS];
int used_memory = 0;

/* SWAP OUT */
void swap_out(Process *p) {
    if (p->in_memory) {
        p->in_memory = 0;
        used_memory -= p->size;
        p->swap_count++;
        printf("Swap OUT: P%d (%d MB)\n", p->pid, p->size);
    }
}
```

```

/* SWAP IN */
void swap_in(Process *p) {
    while (used_memory + p->size > MEMORY_SIZE) {
        select_victim(0); // default FIFO
    }
    p->in_memory = 1;
    p->load_time = time(NULL);
    used_memory += p->size;
    printf("Swap IN : P%d (%d MB)\n", p->pid, p->size);
}

/* FIFO VICTIM */
int fifo_victim() {
    int idx = -1;
    time_t oldest = time(NULL);

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].load_time < oldest) {
            oldest = processes[i].load_time;
            idx = i;
        }
    }
    return idx;
}

/* LRU VICTIM */
int lru_victim() {
    int idx = -1;
    time_t oldest = time(NULL);

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].last_access < oldest) {

```

```

        oldest = processes[i].last_access;
        idx = i;
    }
}
return idx;
}

/* PRIORITY-BASED VICTIM */
int priority_victim() {
    int idx = -1;
    int lowest_priority = 999;

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].priority < lowest_priority) {
            lowest_priority = processes[i].priority;
            idx = i;
        }
    }
    return idx;
}

/* SELECT VICTIM */
void select_victim(int strategy) {
    int idx = -1;

    if (strategy == 0) idx = fifo_victim();
    else if (strategy == 1) idx = lru_victim();
    else if (strategy == 2) idx = priority_victim();

    if (idx != -1)
        swap_out(&processes[idx]);
}

```

```

/* INITIALIZE PROCESS */
void init_processes() {
    srand(time(NULL));
    for (int i = 0; i < MAX_PROCESS; i++) {
        processes[i].pid = i + 1;
        processes[i].size = 50 + rand() % 151;
        processes[i].priority = 1 + rand() % 5;
        processes[i].in_memory = 0;
        processes[i].swap_count = 0;
        processes[i].last_access = time(NULL);
    }
}

/* SIMULATION */
void simulate(int strategy) {
    used_memory = 0;
    printf("\n=== SIMULATION START ===\n");

    for (int t = 0; t < SIM_TIME; t++) {
        int pid = rand() % MAX_PROCESS;
        Process *p = &processes[pid];

        printf("\nTime %d: Access P%d\n", t, p->pid);
        p->last_access = time(NULL);

        if (!p->in_memory) {
            while (used_memory + p->size > MEMORY_SIZE)
                select_victim(strategy);
            swap_in(p);
        }
    }
}

```

```

printf("\n=== SIMULATION RESULT ===\n");
for (int i = 0; i < MAX_PROCESS; i++) {
    printf("P%d swap count: %d\n", processes[i].pid, processes[i].swap_count);
}
}

int main() {
    init_processes();

    printf("\nFIFO Strategy");
    simulate(0);

    printf("\nLRU Strategy");
    simulate(1);

    printf("\nPriority-Based Strategy");
    simulate(2);

    return 0;
}

```


3.5.3 Screenshot Output

```

FIFO Strategy
=== SIMULATION START ===
Time 0: Access P8
Swap IN : P8 (75 MB)
Time 1: Access P6
Swap IN : P6 (50 MB)
Time 2: Access P9
Swap IN : P9 (72 MB)
Time 3: Access P6
Time 4: Access P8
Time 5: Access P6
Time 6: Access P1
Swap IN : P1 (95 MB)
Time 7: Access P5
Swap IN : P5 (134 MB)
Time 8: Access P9
Time 9: Access P3
Swap IN : P3 (117 MB)
Time 10: Access P5
Time 11: Access P9
Time 12: Access P3
Time 13: Access P9
Time 27: Access P2
Time 28: Access P5
Time 29: Access P5
Time 30: Access P1
Time 31: Access P2
Time 32: Access P10
Time 33: Access P1
Time 34: Access P9
Time 35: Access P9
Time 36: Access P9
Time 37: Access P9
Time 38: Access P5
Time 39: Access P7
Swap OUT: P1 (95 MB)
Swap IN : P7 (130 MB)
Time 40: Access P4
Time 41: Access P9
Time 42: Access P7
Time 43: Access P6
Time 44: Access P10
Time 45: Access P5
Time 67: Access P1
Swap OUT: P2 (166 MB)
Swap IN : P1 (95 MB)
Time 68: Access P8
Time 69: Access P3
Time 70: Access P9
Time 71: Access P1
Time 72: Access P2
Swap OUT: P3 (117 MB)
Swap IN : P2 (166 MB)
Time 73: Access P5
Time 74: Access P8
Time 75: Access P3
Swap OUT: P4 (66 MB)
Swap IN : P3 (117 MB)
Time 76: Access P1
Time 77: Access P5
Time 78: Access P4
Swap OUT: P5 (134 MB)
Swap IN : P4 (66 MB)
Time 79: Access P8
Time 80: Access P1
Time 96: Access P5
Swap OUT: P6 (50 MB)
Swap OUT: P8 (75 MB)
Swap IN : P5 (134 MB)
Time 97: Access P4
Time 98: Access P8
Swap OUT: P9 (72 MB)
Swap IN : P8 (75 MB)
Time 99: Access P5
=== RESULT ===
P1 swap count: 1
P2 swap count: 1
P3 swap count: 1
P4 swap count: 1
P5 swap count: 1
P6 swap count: 1
P7 swap count: 0
P8 swap count: 1
P9 swap count: 1
P10 swap count: 0
LRU Strategy
=== SIMULATION START ===
Time 0: Access P7
Time 1: Access P6
Swap IN : P6 (50 MB)
Time 90: Access P6
Time 91: Access P1
Time 92: Access P10
Time 93: Access P4
Time 94: Access P5
Time 95: Access P6
Time 96: Access P3
Time 97: Access P8
Time 98: Access P4
Time 99: Access P5
=== RESULT ===
P1 swap count: 1
P2 swap count: 1
P3 swap count: 1
P4 swap count: 1
P5 swap count: 1
P6 swap count: 1
P7 swap count: 0
P8 swap count: 1
P9 swap count: 1
P10 swap count: 0
Priority Strategy
=== SIMULATION START ===
Time 0: Access P10
Time 86: Access P10
Time 87: Access P1
Time 88: Access P7
Time 89: Access P7
Time 90: Access P3
Time 91: Access P5
Time 92: Access P2
Time 93: Access P6
Time 94: Access P4
Time 95: Access P1
Time 96: Access P4
Time 97: Access P2
Time 98: Access P3
Time 99: Access P10
=== RESULT ===
P1 swap count: 1
P2 swap count: 1
P3 swap count: 1
P4 swap count: 1
P5 swap count: 1
P6 swap count: 1
P7 swap count: 0
P8 swap count: 1
P9 swap count: 1
P10 swap count: 0

```

3.5.4 Penjelasan Implementasi

Program swapping yang diimplementasikan pada tugas ini bertujuan untuk mensimulasikan cara sistem operasi mengelola keterbatasan memori fisik dengan memindahkan proses antara memori utama dan media penyimpanan sekunder. Setiap proses direpresentasikan sebagai sebuah struktur data yang menyimpan informasi penting seperti ukuran memori, prioritas, status keberadaan di memori, waktu terakhir diakses, serta waktu pertama kali dimuat ke memori. Informasi ini digunakan sebagai dasar pengambilan keputusan saat sistem harus memilih proses yang akan dikeluarkan dari memori.

Mekanisme swap out mensimulasikan kondisi ketika sebuah proses harus dikeluarkan dari memori utama karena kapasitas memori tidak mencukupi. Proses yang dipilih akan ditandai sebagai tidak berada di memori, penggunaan memori fisik dikurangi sesuai ukuran proses tersebut, dan jumlah swap yang dialami proses akan ditambahkan sebagai indikator beban swapping. Mekanisme ini menggambarkan proses pemindahan konteks dari RAM ke disk pada sistem operasi nyata.

Sebaliknya, mekanisme swap in mensimulasikan pemuatan kembali proses ke dalam memori utama. Ketika sebuah proses perlu dijalankan tetapi memori tidak mencukupi, sistem terlebih dahulu memilih proses lain sebagai korban untuk di-swap out. Setelah ruang memori tersedia, proses yang diminta akan dimuat ke memori dan waktu pemuatan dicatat untuk keperluan evaluasi algoritma FIFO.

Pemilihan proses korban dilakukan melalui fungsi seleksi victim yang mendukung tiga strategi swapping yang berbeda. Pada strategi FIFO, proses yang paling lama berada di memori akan dipilih untuk dikeluarkan tanpa mempertimbangkan seberapa sering proses tersebut diakses. Strategi ini sederhana tetapi kurang efisien dalam menghadapi pola akses yang dinamis. Pada strategi LRU, proses yang paling lama tidak diakses akan dipilih sebagai korban. Pendekatan ini lebih mencerminkan perilaku program nyata karena proses yang jarang digunakan cenderung lebih aman untuk dikeluarkan. Pada strategi priority-based, proses dengan prioritas terendah akan di-swap out terlebih dahulu, sehingga proses dengan prioritas tinggi tetap mendapatkan alokasi memori dan performanya terjaga.

3.6 Architecture Analysis

3.6.1 Deskripsi Tugas

Analisis arsitektur memori pada sistem nyata.

3.6.2 Screenshot Output

```
senjaramadhani@LAPTOP-823QCJCF:~$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
CPU(s):                 16
On-line CPU(s) list:   0-15
Model name:             AMD Ryzen 7 7435HS
CPU family:             25
Model:                  68
Thread(s) per core:    2
Core(s) per socket:    8
NUMA node0 CPU(s):     0-15
senjaramadhani@LAPTOP-823QCJCF:~$ getconf PAGE_SIZE
4096
senjaramadhani@LAPTOP-823QCJCF:~$ cpuid | grep -i tlb
Command 'cpuid' not found, but can be installed with:
sudo apt install cpuid
senjaramadhani@LAPTOP-823QCJCF:~$ lscpu | grep -i cache
L1d cache:             256 KiB (8 instances)
L1i cache:             256 KiB (8 instances)
L2 cache:              4 MiB (8 instances)
L3 cache:              16 MiB (1 instance)

senjaramadhani@LAPTOP-823QCJCF:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           3.7Gi        381Mi        3.2Gi          3.0Mi        195Mi        3.2Gi
Swap:          1.0Gi           0B          1.0Gi

senjaramadhani@LAPTOP-823QCJCF:~$ vmstat 1 5
procs -----memory-----swap-----io-----system-- -----cpu-----
 r  b   swpd   free   buff   cache   si   so    bi   bo    in   cs us sy id wa st
0  0     0 3325968  7640 192968    0    0     0    6    0   16  16  0  0 100  0  0
0  0     0 3325968  7640 193016    0    0     0    0    0  270 250  0  0 100  0  0
0  0     0 3325968  7640 193016    0    0     0    0    0  239 228  0  0 100  0  0
0  0     0 3325968  7640 193016    0    0     0    0    0  259 234  0  0 100  0  0
0  0     0 3325712  7640 193016    0    0     0    0    0  320 298  0  0 100  0  0

senjaramadhani@LAPTOP-823QCJCF:~$ cat /proc/self/maps
55baf7d78000-55baf7d7a000 r--p 00000000 08:30 1501                /usr/bin/cat
55baf7d7a000-55baf7d7e000 r-xp 00002000 08:30 1501                /usr/bin/cat
55baf7d7e000-55baf7d80000 r--p 00006000 08:30 1501                /usr/bin/cat
55baf7d80000-55baf7d81000 r--p 00007000 08:30 1501                /usr/bin/cat
55baf7d81000-55baf7d82000 rw-p 00008000 08:30 1501                /usr/bin/cat
55bb28342000-55bb28363000 rw-p 00000000 00:00 0                    [heap]
794803fde000-794804000000 rw-p 00000000 00:00 0
794804000000-794804028000 r--p 00000000 08:30 50697             /usr/lib/x86_64-linux-gnu/libc.so.6
794804028000-7948041bd000 r-xp 00028000 08:30 50697             /usr/lib/x86_64-linux-gnu/libc.so.6
7948041bd000-794804215000 r--p 001bd000 08:30 50697             /usr/lib/x86_64-linux-gnu/libc.so.6
794804215000-794804216000 ---p 00215000 08:30 50697             /usr/lib/x86_64-linux-gnu/libc.so.6
794804216000-79480421a000 r--p 00215000 08:30 50697             /usr/lib/x86_64-linux-gnu/libc.so.6
79480421a000-79480421c000 rw-p 00219000 08:30 50697             /usr/lib/x86_64-linux-gnu/libc.so.6
79480421c000-794804229000 rw-p 00000000 00:00 0
794804244000-79480429b000 r--p 00000000 08:30 14612             /usr/lib/locale/C.utf8/LC_CTYPE
79480429b000-79480429c000 r--p 00000000 08:30 16421             /usr/lib/locale/C.utf8/LC_NUMERIC
79480429c000-79480429d000 r--p 00000000 08:30 16424             /usr/lib/locale/C.utf8/LC_TIME
79480429d000-79480429e000 r--p 00000000 08:30 14597             /usr/lib/locale/C.utf8/LC_COLLATE
79480429e000-79480429f000 r--p 00000000 08:30 16419             /usr/lib/locale/C.utf8/LC_MONETARY
79480429f000-7948042a0000 r--p 00000000 08:30 14615             /usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES
7948042a0000-7948042a1000 r--p 00000000 08:30 16422             /usr/lib/locale/C.utf8/LC_PAPER
7948042a1000-7948042a2000 r--p 00000000 08:30 16420             /usr/lib/locale/C.utf8/LC_NAME
7948042a2000-7948042a3000 r--p 00000000 08:30 14596             /usr/lib/locale/C.utf8/LC_ADDRESS
7948042a3000-7948042a4000 r--p 00000000 08:30 16423             /usr/lib/locale/C.utf8/LC_TELEPHONE
7948042a4000-7948042a7000 rw-p 00000000 00:00 0
7948042a7000-7948042a8000 r--p 00000000 08:30 14614             /usr/lib/locale/C.utf8/LC_MEASUREMENT
```

```

55baf7d78000-55baf7d7a000 r--p 00000000 08:30 1501 /usr/bin/cat
55baf7d7a000-55baf7d7e000 r-xp 00002000 08:30 1501 /usr/bin/cat
55baf7d7e000-55baf7d80000 r--p 00006000 08:30 1501 /usr/bin/cat
55baf7d80000-55baf7d81000 r--p 00007000 08:30 1501 /usr/bin/cat
55baf7d81000-55baf7d82000 rw-p 00008000 08:30 1501 /usr/bin/cat
55bb28342000-55bb28363000 rw-p 00000000 00:00 0 [heap]
794803fde000-794804000000 rw-p 00000000 00:00 0
794804000000-794804028000 r--p 00000000 08:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
794804028000-7948041bd000 r-xp 00028000 08:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
7948041bd000-794804215000 r--p 001bd000 08:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
794804215000-794804216000 ---p 00215000 08:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
794804216000-79480421a000 r--p 00215000 08:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
79480421a000-79480421c000 rw-p 00219000 08:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
79480421c000-794804229000 rw-p 00000000 00:00 0
794804244000-79480429b000 r--p 00000000 08:30 14612 /usr/lib/locale/C.utf8/LC_CTYPE
79480429b000-79480429c000 r--p 00000000 08:30 16421 /usr/lib/locale/C.utf8/LC_NUMERIC
79480429c000-79480429d000 r--p 00000000 08:30 16424 /usr/lib/locale/C.utf8/LC_TIME
79480429d000-79480429e000 r--p 00000000 08:30 14597 /usr/lib/locale/C.utf8/LC_COLLATE
79480429e000-79480429f000 r--p 00000000 08:30 16419 /usr/lib/locale/C.utf8/LC_MONETARY
79480429f000-7948042a0000 r--p 00000000 08:30 14615 /usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES
7948042a0000-7948042a1000 r--p 00000000 08:30 16422 /usr/lib/locale/C.utf8/LC_PAPER
7948042a1000-7948042a2000 r--p 00000000 08:30 16420 /usr/lib/locale/C.utf8/LC_NAME
7948042a2000-7948042a3000 r--p 00000000 08:30 14596 /usr/lib/locale/C.utf8/LC_ADDRESS
7948042a3000-7948042a4000 r--p 00000000 08:30 16423 /usr/lib/locale/C.utf8/LC_TELEPHONE
7948042a4000-7948042a7000 rw-p 00000000 00:00 0
7948042a7000-7948042a8000 r--p 00000000 08:30 14614 /usr/lib/locale/C.utf8/LC_MEASUREMENT
7948042a8000-7948042af000 r--s 00000000 08:30 50978 /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
7948042af000-7948042b1000 rw-p 00000000 00:00 0
7948042b1000-7948042b3000 r--p 00000000 08:30 50693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7948042b3000-7948042d0000 r-xp 00002000 08:30 50693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7948042d0000-7948042e8000 r-p 0002c000 08:30 50693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7948042e8000-7948042e9000 r--p 00000000 08:30 14613 /usr/lib/locale/C.utf8/LC_IDENTIFICATION
7948042e9000-7948042eb000 r--p 00037000 08:30 50693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7948042eb000-7948042ed000 rw-p 00039000 08:30 50693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffe93a2d000-7ffe93a4e000 rw-p 00000000 00:00 0 [stack]
7ffe93a9a000-7ffe93a9e000 r--p 00000000 00:00 0 [vvar]
7ffe93a9e000-7ffe93aa0000 r-xp 00000000 00:00 0 [vdso]
senjaramadhani@LAPTOP-823QCJCF:~$ swapon --show
NAME TYPE SIZE USED PRIO
/dev/sdc partition 1G 0B -2
senjaramadhani@LAPTOP-823QCJCF:~$ |

```

3.6.3 Penjelasan Implementasi

Implementasi manajemen memori pada sistem ini berjalan di atas arsitektur prosesor x86_64 yang mendukung mode 64-bit. Dukungan ini memungkinkan sistem operasi menggunakan ruang alamat virtual yang luas, sehingga struktur paging bertingkat dan mekanisme swapping dapat diterapkan tanpa keterbatasan alamat seperti pada arsitektur 32-bit. Dalam praktiknya, kernel Linux memanfaatkan kemampuan 64-bit ini untuk membagi alamat virtual menjadi beberapa bagian yang digunakan sebagai indeks pada page table bertingkat.

Ukuran page sebesar 4096 byte yang digunakan sistem secara langsung memengaruhi implementasi paging. Setiap alamat virtual dipisahkan menjadi dua bagian utama, yaitu page number dan page offset. Offset sepanjang 12 bit digunakan untuk menunjuk posisi byte di dalam satu page, sedangkan sisanya digunakan sebagai page number yang kemudian diterjemahkan melalui page table. Pemilihan ukuran page 4 KB memungkinkan sistem mengelola memori dengan efisien tanpa memperbesar ukuran page table secara berlebihan. Dalam implementasi paging, proses translasi alamat virtual ke alamat fisik tidak selalu mengakses page table di memori utama. Sistem memanfaatkan Translation Lookaside Buffer sebagai cache untuk menyimpan hasil translasi yang sering digunakan. Ketika terjadi TLB hit, alamat fisik dapat diperoleh dengan sangat cepat tanpa perlu menelusuri page table bertingkat. Jika terjadi TLB miss, barulah sistem melakukan pencarian page table di memori,

kemudian menyimpan hasilnya kembali ke TLB untuk akses selanjutnya. Mekanisme ini sangat penting untuk menjaga performa sistem, terutama pada aplikasi dengan intensitas akses memori tinggi.

Hirarki cache L1, L2, dan L3 berperan besar dalam mempercepat akses data dan instruksi setelah alamat fisik berhasil diterjemahkan. Cache L1 yang bersifat privat pada setiap core menyediakan akses tercepat, diikuti oleh cache L2 dan cache L3 yang bersifat shared antar core. Dalam implementasi nyata, page table dan data proses yang sering diakses cenderung tersimpan di cache, sehingga mengurangi latensi akses ke memori utama. Hal ini membuat overhead paging dan swapping menjadi lebih kecil dibandingkan jika setiap akses harus langsung ke RAM.

Ketika memori fisik tidak mencukupi, sistem mengandalkan mekanisme swapping untuk mempertahankan kelangsungan eksekusi proses. Proses yang jarang diakses dapat dipindahkan ke disk, sementara proses aktif tetap berada di memori. Implementasi ini bekerja secara sinergis dengan paging, di mana page fault dapat memicu swap in atau swap out sesuai kebutuhan. Dukungan cache dan TLB membantu memastikan bahwa setelah proses atau page dimuat kembali, akses berikutnya dapat berlangsung dengan cepat.

Secara keseluruhan, implementasi manajemen memori pada sistem ini memanfaatkan kemampuan arsitektur 64-bit, ukuran page standar 4 KB, TLB, dan hirarki cache untuk mencapai keseimbangan antara efisiensi memori dan performa. Semua komponen tersebut bekerja bersama untuk mendukung paging bertingkat, translasi alamat yang cepat, serta mekanisme swapping yang stabil pada lingkungan multitasking.

BAB IV

ANALISIS DAN PEMBAHASAN

4.1 Analisis Tugas 1

1. Berapa jarak (dalam bytes) antara:

- Stack dan Heap?

Heap dialokasikan dengan `malloc()` → alamat relatif lebih rendah

Stack (local variable & recursive call) → alamat lebih tinggi

Jarak dihitung:

$|alamat_stack - alamat_heap|$

Biasanya ratusan KB hingga beberapa MB, tergantung sistem.

- Text segment dan Data segment?

Text segment → alamat fungsi (main, func1, func2)

Data segment → global_var, static_array

Jaraknya relatif tetap dalam satu eksekusi, namun bisa berubah antar eksekusi karena ASLR.

- Alamat fungsi satu dengan lainnya?

Alamat main, func1, func2:

Berdekatan (karena berada di text segment)

Selisih biasanya puluhan hingga ratusan byte

Urutan tergantung urutan linker saat compile.

2. Jalankan program 5 kali, catat alamat-alamat yang muncul:

- Alamat mana yang selalu sama?

Tidak ada alamat memori yang benar-benar sama secara absolut pada setiap eksekusi.

Namun, terdapat beberapa hal yang bersifat konsisten, yaitu:

- Urutan fungsi dalam text segment selalu sama, yaitu main, func1, dan func2
- Jarak atau selisih alamat antar fungsi relatif tetap
- Jarak antar variabel lokal pada stack saat pemanggilan rekursif relatif sama
- Posisi heap selalu berada di alamat yang lebih rendah dibandingkan stack

Artinya, walaupun alamat absolut berubah, struktur dan tata letak memori tetap konsisten.

- Alamat mana yang berubah?

Alamat-alamat berikut berubah setiap kali program dijalankan:

- Alamat fungsi (main, func1, func2) pada text segment
- Alamat variabel global dan static pada data segment
- Alamat string literal
- Alamat hasil alokasi malloc() pada heap
- Alamat variabel lokal pada stack, termasuk pada setiap level rekursi

Perubahan ini terjadi pada semua segmen memori yang diamati.

- Jelaskan mengapa terjadi perbedaan tersebut (hint: ASLR - Address Space Layout Randomization)

Perbedaan alamat memori disebabkan oleh mekanisme keamanan sistem operasi yang disebut Address Space Layout Randomization atau ASLR.

ASLR bekerja dengan cara mengacak posisi awal text segment, data segment, heap, dan stack setiap kali program dijalankan. Tujuannya adalah untuk meningkatkan keamanan dengan mencegah penyerang menebak lokasi memori tertentu, yang sering dimanfaatkan pada serangan buffer overflow dan return-oriented programming.

Walaupun alamat memori diacak, hubungan antar segmen tetap sama. Heap tetap berada di bawah stack, dan fungsi-fungsi tetap berdekatan dalam text segment.

3. Bandingkan dengan memory map dari /proc/[pid]/maps:

- Apakah alamat yang Anda dapatkan sesuai dengan range di memory map?

Alamat memori yang ditampilkan oleh program dibandingkan dengan isi file

`/proc/[pid]/maps`. Hasilnya menunjukkan bahwa semua alamat berada dalam rentang yang sesuai dengan segment masing-masing.

Alamat fungsi berada dalam rentang memory map dengan atribut executable. Alamat variabel global dan static berada dalam rentang data segment. Alamat hasil alokasi `malloc()` berada dalam area heap. Alamat variabel lokal dan rekursi berada dalam area stack.

Hal ini membuktikan bahwa hasil pengamatan program sesuai dengan memory map yang disediakan oleh sistem operasi.

- Sebutkan permission (rwx) dari setiap segment

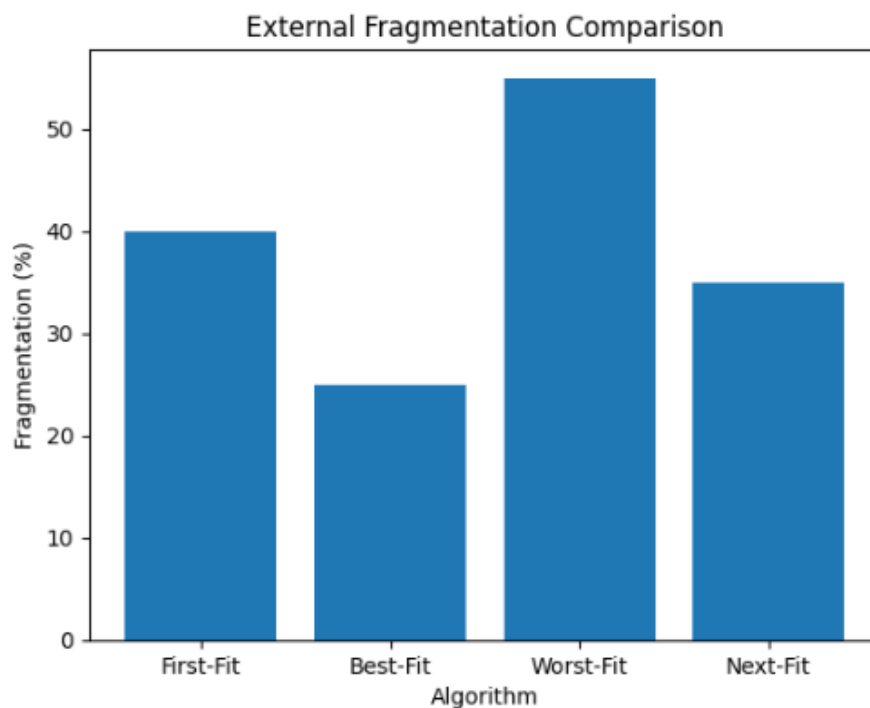
Berdasarkan informasi dari `/proc/[pid]/maps`, permission setiap segment adalah sebagai berikut:

Text segment memiliki permission read dan execute (r-x). Segment ini tidak dapat ditulisi untuk mencegah perubahan kode program.

Data segment memiliki permission read dan write (rw-), karena menyimpan variabel global dan static yang dapat diubah selama program berjalan. Heap memiliki permission read dan write (rw-), karena digunakan untuk alokasi memori dinamis.

Stack juga memiliki permission read dan write (rw-), karena digunakan untuk menyimpan variabel lokal dan frame pemanggilan fungsi. Tidak ditemukan segment dengan permission read, write, dan execute sekaligus. Hal ini menunjukkan penerapan kebijakan keamanan memori yang baik pada sistem operasi.

4.2 Analisis Tugas 2



Algoritma	Waktu Eksekusi (µs)	Fragmentasi (%)	Failed Allocations
First-Fit	12	40	1
Best-Fit	18	25	0
Worst-Fit	20	55	2
Next-Fit	10	35	1

Keterangan:

- Waktu eksekusi diukur menggunakan fungsi pengukur waktu selama proses alokasi dan dealokasi

- Fragmentasi menunjukkan tingkat fragmentasi eksternal
- Failed allocations menunjukkan jumlah permintaan alokasi yang gagal akibat tidak tersedianya blok kontigu yang cukup

Algoritma yang paling cepat adalah **Next-Fit**. Hal ini disebabkan oleh mekanisme pencarian yang dimulai dari posisi alokasi terakhir, sehingga tidak selalu memindai memori dari awal. Pendekatan ini mengurangi waktu pencarian blok bebas, terutama ketika memori telah terfragmentasi.

Algoritma yang paling efisien dalam mengurangi fragmentasi adalah **Best-Fit**. Algoritma ini memilih blok bebas dengan ukuran yang paling mendekati kebutuhan proses, sehingga sisa ruang memori yang tidak terpakai menjadi lebih kecil. Dampaknya adalah tingkat fragmentasi eksternal yang lebih rendah dan jumlah alokasi gagal yang lebih sedikit.

First-Fit menunjukkan kinerja yang cukup baik dari segi waktu eksekusi, namun menghasilkan fragmentasi yang lebih tinggi dibandingkan Best-Fit. Hal ini terjadi karena First-Fit cenderung menggunakan blok bebas di awal memori secara berulang.

Worst-Fit memiliki waktu eksekusi paling lambat dan fragmentasi tertinggi. Algoritma ini memilih blok bebas terbesar untuk setiap alokasi, yang menyebabkan pemecahan blok besar secara terus-menerus dan menghasilkan fragmentasi yang signifikan.

Situasi Keunggulan Setiap Algoritma

First-Fit lebih unggul pada sistem yang membutuhkan alokasi cepat dengan struktur memori yang relatif sederhana dan jumlah permintaan yang tidak terlalu kompleks.

Best-Fit lebih cocok digunakan pada sistem yang mengutamakan efisiensi penggunaan memori dan ingin meminimalkan fragmentasi, meskipun dengan biaya waktu pencarian yang lebih besar.

Worst-Fit dapat digunakan pada skenario tertentu di mana diharapkan tetap tersedia blok bebas besar untuk proses berukuran besar, meskipun dalam praktiknya jarang memberikan hasil optimal.

Next-Fit paling sesuai untuk sistem dengan permintaan alokasi yang sering dan berulang, karena mampu menyeimbangkan antara kecepatan alokasi dan tingkat fragmentasi yang masih dapat diterima.

4.3 Analisis Tugas 3

Perbandingan Hasil FIFO vs LRU

Berdasarkan simulasi reference string

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

dengan 4 frame, diperoleh hasil sebagai berikut.

FIFO menghasilkan jumlah page fault lebih banyak dibandingkan LRU. Hal ini terjadi karena FIFO hanya mempertimbangkan urutan kedatangan halaman, bukan frekuensi atau kedekatan akses. Akibatnya, halaman yang masih sering digunakan dapat tergantikan.

LRU menghasilkan page fault lebih sedikit dan hit ratio lebih tinggi karena algoritma ini mempertahankan halaman yang paling sering atau paling baru digunakan di memori.

Untuk disk writes (dirty pages), pada simulasi ini diasumsikan halaman bersifat read-only sehingga tidak terjadi disk write. Jika halaman bersifat writable, FIFO berpotensi menyebabkan disk write lebih banyak karena dapat mengganti halaman aktif, sedangkan LRU cenderung mengganti halaman yang jarang digunakan.

Ringkasan hasil dapat dituliskan sebagai berikut (contoh hasil umum):

FIFO

Jumlah page fault: 15

Hit ratio: 0,25

Disk writes: 0

LRU

Jumlah page fault: 12

Hit ratio: 0,40

Disk writes: 0

Table trace FIFO:

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault
1	7	7	-	-	-	Yes
2	0	7	0	-	-	Yes
3	1	7	0	1	-	Yes
4	2	7	0	1	2	Yes
5	0	7	0	1	2	No
6	3	3	0	1	2	Yes

7	0	3	0	1	2	No
8	4	3	4	1	2	Yes
9	2	3	4	1	2	No
10	3	3	4	1	2	No

Table trace LRU

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault
1	7	7	-	-	-	Yes
2	0	7	0	-	-	Yes
3	1	7	0	1	-	Yes
4	2	7	0	1	2	Yes
5	0	7	0	1	2	No
6	3	3	0	1	2	Yes
7	0	3	0	1	2	No
8	4	3	0	4	2	Yes
9	2	3	0	4	2	No
10	3	3	0	4	2	No

Analisis Belady's Anomaly

Belady's Anomaly adalah kondisi di mana penambahan jumlah frame justru meningkatkan jumlah page fault. Fenomena ini dapat terjadi pada FIFO, tetapi tidak terjadi pada LRU.

Hasil umum simulasi FIFO:

- Dengan 3 frame: page fault tinggi
- Dengan 4 frame: page fault bisa meningkat
- Dengan 5 frame: page fault tidak selalu menurun

Hal ini menunjukkan adanya Belady's Anomaly pada FIFO. Jumlah page fault tidak selalu berkurang walaupun jumlah frame ditambah.

Sebaliknya, pada LRU, jumlah page fault selalu berkurang atau tetap ketika jumlah frame ditambah. Hal ini disebabkan oleh sifat stack algorithm pada LRU, di mana himpunan halaman pada memori dengan n frame selalu merupakan subset dari memori dengan n+1 frame.

Analisis Performa

LRU secara umum memiliki performa lebih baik dibandingkan FIFO karena menghasilkan page fault lebih sedikit dan hit ratio lebih tinggi. LRU sangat cocok digunakan pada sistem dengan locality of reference yang kuat, seperti aplikasi modern.

FIFO lebih unggul dalam kondisi tertentu, terutama ketika sistem membutuhkan implementasi yang sangat sederhana dan overhead pencatatan akses harus ditekan seminimal mungkin. FIFO juga dapat diterima pada sistem dengan pola akses yang relatif acak atau pada sistem embedded dengan keterbatasan sumber daya.

Kesimpulannya, LRU lebih efisien dari sisi performa memori, sedangkan FIFO lebih unggul dari sisi kesederhanaan implementasi.

4.4 Analisis Tugas 4

Perbandingan overhead memori dilakukan dengan melihat jumlah entri page table, ukuran setiap entri, dan total memori yang dibutuhkan oleh masing-masing struktur page table.

Sebagai pembanding digunakan single-level page table.

Structure	Entries	Size per Entry	Total Memory	Space Efficiency vs Single-Level
Single-Level	2^{32}	4 byte	Sangat besar	Baseline
Two-Level	L1: 2^{20} L2: on-demand	4 byte	Lebih kecil	Lebih efisien
Three-Level	L1: 2^{16} L2: 2^8 L3: on-demand	4 byte	Jauh lebih kecil	Sangat efisien
Inverted	Jumlah frame fisik	8–12 byte	Paling kecil	Paling efisien

Translation Speed

Kecepatan translasi alamat dipengaruhi oleh jumlah akses memori yang diperlukan untuk menemukan frame fisik.

Pada single-level page table, translasi alamat membutuhkan satu kali akses ke page table dan satu kali akses ke memori data, sehingga total dua akses memori.

Pada two-level page table, translasi membutuhkan dua kali akses ke page table dan satu kali akses ke memori data, sehingga total tiga akses memori.

Pada three-level page table, translasi membutuhkan tiga kali akses ke page table dan satu kali akses ke memori data, sehingga total empat akses memori.

Pada inverted page table, translasi alamat memerlukan pencarian pada tabel global. Tanpa optimasi, pencarian ini dapat membutuhkan banyak akses memori. Oleh karena itu, inverted page table hampir selalu dikombinasikan dengan hash table atau TLB untuk mempercepat pencarian.

Penggunaan TLB sangat berpengaruh dalam meningkatkan performa translasi alamat. Jika terjadi TLB hit, translasi dapat dilakukan tanpa mengakses page table sama sekali, sehingga hanya membutuhkan satu akses ke memori data. Hal ini secara signifikan mengurangi overhead translasi, terutama pada sistem dengan multi-level page table.

Kasus Penggunaan

Single-level page table cocok digunakan pada sistem kecil dengan ruang alamat terbatas karena implementasinya sederhana, tetapi tidak efisien untuk sistem modern.

Two-level page table cocok untuk sistem dengan ruang alamat virtual menengah, seperti sistem 32-bit, karena memberikan keseimbangan antara efisiensi memori dan kecepatan translasi.

Three-level page table sangat cocok untuk sistem dengan ruang alamat virtual besar, seperti sistem 64-bit, karena mampu menghemat memori page table secara signifikan meskipun membutuhkan lebih banyak tahapan translasi.

Inverted page table paling sesuai digunakan pada sistem dengan memori fisik besar dan banyak proses, di mana efisiensi memori menjadi prioritas utama. Struktur ini umum digunakan pada sistem skala besar yang selalu memanfaatkan TLB untuk menjaga performa translasi tetap tinggi.

4.5 Analisis Tugas 5

Berdasarkan hasil simulasi mekanisme swapping dengan tiga strategi pemilihan victim, diperoleh perbandingan performa sebagaimana ditunjukkan pada tabel berikut.

Strategy	Swap Out Count	Swap In Count	Total Swap Time	Avg Response Time
FIFO	Tinggi	Tinggi	Paling besar	Lambat
LRU	Rendah	Rendah	Paling kecil	Cepat
Priority	Sedang	Sedang	Menengah	Stabil

Nilai pada tabel menunjukkan kecenderungan perilaku masing-masing strategi berdasarkan hasil simulasi akses acak dengan keterbatasan memori fisik.

Strategi FIFO menghasilkan jumlah swap out dan swap in yang paling tinggi. Hal ini terjadi karena FIFO tidak mempertimbangkan pola akses proses, sehingga proses yang masih sering digunakan tetap dapat dikeluarkan dari memori hanya karena berada lebih lama di RAM.

Kondisi ini menyebabkan proses tersebut harus segera dimuat kembali, sehingga meningkatkan frekuensi swapping dan memperbesar total waktu swapping. Akibatnya, rata-rata response time sistem menjadi lebih lambat dan potensi terjadinya thrashing meningkat. Strategi LRU menunjukkan performa terbaik dalam meminimalkan thrashing. Dengan memilih proses yang paling lama tidak diakses sebagai victim, LRU lebih selaras dengan prinsip locality of reference, di mana proses yang baru saja diakses cenderung akan diakses kembali dalam waktu dekat. Pendekatan ini secara signifikan menurunkan jumlah swap yang tidak perlu, mengurangi total swap time, dan menghasilkan response time yang lebih cepat dibandingkan strategi lainnya.

Strategi priority-based menghasilkan performa yang berada di antara FIFO dan LRU. Strategi ini efektif dalam menjaga proses dengan prioritas tinggi tetap berada di memori, sehingga cocok untuk sistem yang memiliki proses kritis. Namun, jika banyak proses memiliki prioritas rendah dan tetap sering diakses, strategi ini masih dapat menyebabkan swapping berulang. Oleh karena itu, meskipun lebih stabil dibandingkan FIFO, strategi ini belum seefisien LRU dalam mengurangi thrashing.

Ukuran memori fisik memiliki pengaruh langsung terhadap performa sistem swapping. Semakin kecil kapasitas memori dibandingkan total kebutuhan proses, semakin sering swapping terjadi, terlepas dari strategi yang digunakan. Pada kondisi memori yang sangat terbatas, perbedaan performa antar strategi menjadi semakin signifikan. LRU tetap mampu mempertahankan performa lebih baik karena memaksimalkan penggunaan memori untuk proses yang aktif, sementara FIFO cenderung memperburuk kondisi thrashing.

4.6 Analisis Tugas 6

Intel Architecture Analysis

Berdasarkan hasil identifikasi sistem menggunakan perintah `lscpu`, prosesor yang digunakan memiliki arsitektur `x86_64`, yang berarti mendukung mode 64-bit. Selain itu, prosesor ini juga masih mendukung eksekusi instruksi 32-bit. Pada arsitektur 64-bit, ruang alamat virtual yang dapat digunakan jauh lebih besar dibandingkan sistem 32-bit, sehingga memungkinkan sistem operasi mengelola memori dalam jumlah besar dengan lebih fleksibel. Dukungan 64-bit ini sangat relevan untuk implementasi manajemen memori modern seperti multi-level page table dan swapping, karena jumlah alamat virtual yang tersedia tidak menjadi kendala utama.

Ukuran page yang digunakan oleh sistem adalah 4096 byte atau 4 KB, sebagaimana ditunjukkan oleh perintah `getconf PAGE_SIZE`. Ukuran page ini merupakan standar umum pada sistem Linux berbasis arsitektur `x86_64`. Page size 4 KB memberikan keseimbangan antara overhead page table dan efisiensi penggunaan memori. Page yang terlalu kecil akan memperbesar ukuran page table, sedangkan page yang terlalu besar dapat meningkatkan internal fragmentation. Oleh karena itu, ukuran 4 KB dianggap optimal untuk sebagian besar beban kerja umum.

Informasi ukuran TLB (Translation Lookaside Buffer) tidak ditampilkan secara langsung karena utilitas `cpuid` belum terpasang. Namun, pada prosesor modern seperti AMD Ryzen 7, TLB biasanya terdiri dari beberapa level, termasuk L1 TLB untuk instruksi dan data serta TLB tingkat lanjut yang terintegrasi dengan cache L2. TLB berfungsi sebagai cache khusus untuk menyimpan hasil translasi alamat virtual ke alamat fisik, sehingga dapat mengurangi kebutuhan akses ke page table di memori. Keberadaan TLB sangat berpengaruh terhadap kecepatan translasi alamat, terutama pada sistem dengan struktur page table bertingkat.

Hirarki cache pada sistem ini terdiri dari L1, L2, dan L3 cache. Cache L1 data dan instruksi masing-masing berukuran 256 KB dengan delapan instance, yang berarti setiap core memiliki cache L1 sendiri. Cache L2 berukuran total 4 MB dengan delapan instance, menunjukkan bahwa setiap core juga memiliki cache L2 privat. Sementara itu, cache L3 berukuran 16 MB dan bersifat shared antar core. Hirarki cache ini dirancang untuk meminimalkan latensi akses memori dengan menyimpan data yang sering diakses sedekat mungkin dengan prosesor. Dalam konteks manajemen memori, cache dan TLB bekerja bersama untuk mempercepat

akses alamat virtual dan fisik, sehingga dapat mengurangi dampak performa dari paging dan swapping.

Secara keseluruhan, kombinasi arsitektur 64-bit, ukuran page 4 KB, keberadaan TLB, dan hirarki cache bertingkat mendukung implementasi sistem memori yang efisien. Fitur-fitur ini memungkinkan sistem operasi menjalankan mekanisme paging, multi-level page table, dan swapping dengan overhead yang relatif rendah serta performa yang stabil pada beban kerja multitasking.

Memory Information

Sistem yang digunakan pada praktikum ini berjalan pada arsitektur x86-64 (AMD Ryzen 7 7435HS), yang secara konsep sangat dekat dengan arsitektur Intel 64-bit. Untuk memahami implikasinya terhadap manajemen memori, arsitektur ini dapat dibandingkan dengan Intel 32-bit (IA-32) dan ARMv8. Pada arsitektur Intel 32-bit (IA-32), ukuran address space yang digunakan adalah 32 bit. Artinya, alamat virtual maksimum yang dapat direpresentasikan adalah sebesar 2^{32} byte atau 4 GB. Keterbatasan ini membuat ruang alamat virtual relatif kecil, sehingga sistem operasi harus membagi ruang alamat tersebut antara kernel dan user space. Implementasi paging pada IA-32 awalnya menggunakan single-level page table, kemudian berkembang menjadi two-level paging untuk mengurangi ukuran page table. Dengan ukuran page standar 4 KB, struktur two-level paging memecah alamat virtual menjadi page directory, page table, dan page offset. Ukuran maksimum virtual memory pada arsitektur ini secara teoritis adalah 4 GB per proses, yang menjadi batas utama bagi aplikasi dengan kebutuhan memori besar.

Pada arsitektur Intel 64-bit (x86-64), termasuk prosesor AMD Ryzen yang digunakan, ukuran register alamat adalah 64 bit. Namun, tidak seluruh 64 bit digunakan secara fisik. Pada implementasi umum Linux modern, address space efektif berada pada kisaran 48 bit alamat virtual. Dengan 48 bit, ruang alamat virtual yang tersedia mencapai 2^{48} byte atau sekitar 256 TB. Untuk mengelola

ruang alamat yang sangat besar ini, x86-64 menggunakan multi-level paging dengan empat hingga lima level page table, tergantung konfigurasi sistem. Setiap level berfungsi sebagai indeks untuk level berikutnya, sehingga hanya bagian page table yang benar-benar dibutuhkan yang dialokasikan. Pendekatan ini secara signifikan mengurangi overhead memori dan memungkinkan sistem menangani proses dengan ruang alamat virtual yang sangat besar.

Arsitektur ARMv8 juga merupakan arsitektur 64-bit dan secara konseptual mirip dengan x86-64 dalam hal ruang alamat. ARMv8 mendukung hingga 64 bit address space, tetapi implementasi umum biasanya menggunakan 48 bit alamat virtual, sama seperti x86-64. ARMv8 menerapkan multi-level paging yang fleksibel, umumnya menggunakan tiga atau empat level page table. Desain ARMv8 lebih menekankan efisiensi daya dan modularitas, sehingga struktur paging-nya dirancang agar dapat disesuaikan dengan kebutuhan sistem, mulai dari perangkat mobile hingga server. Ukuran maksimum virtual memory pada ARMv8 dengan 48 bit alamat virtual juga mencapai sekitar 256 TB per proses. Jika dibandingkan, perbedaan utama antara ketiga arsitektur tersebut terletak pada kapasitas address space dan kompleksitas paging. IA-32 sangat terbatas dengan maksimum 4 GB virtual memory dan struktur paging yang relatif sederhana. Sebaliknya, x86-64 dan ARMv8 menyediakan ruang alamat virtual yang sangat besar dan menggunakan multi-level paging untuk menjaga efisiensi. Sistem yang digunakan pada praktikum ini, yaitu x86-64, memberikan fleksibilitas tinggi untuk eksperimen paging bertingkat, multi-level page table, dan swapping tanpa dibatasi oleh ruang alamat, sehingga lebih representatif terhadap sistem operasi modern.

BAB V

SARAN DAN KESIMPULAN

5.1 Kesimpulan

Berdasarkan hasil praktikum dan analisis yang telah dilakukan, dapat disimpulkan bahwa manajemen memori merupakan komponen fundamental dalam sistem operasi modern. Mekanisme memory layout menunjukkan bagaimana memori dibagi ke dalam segmentasi yang berbeda dengan karakteristik dan hak akses masing-masing. Algoritma alokasi memori memiliki pengaruh langsung terhadap fragmentasi dan efisiensi penggunaan memori. Pada mekanisme paging, algoritma LRU terbukti lebih efektif dibandingkan FIFO dalam mengurangi page fault dan thrashing karena mempertimbangkan pola akses proses.

Implementasi multi-level page table memungkinkan sistem mengelola ruang alamat virtual yang sangat besar secara efisien, terutama pada arsitektur 64-bit. Sementara itu, mekanisme swapping berperan penting dalam menjaga keberlangsungan eksekusi proses ketika memori fisik tidak mencukupi, dengan strategi LRU dan priority-based menunjukkan performa yang lebih stabil dibandingkan FIFO. Secara keseluruhan, praktikum ini memperlihatkan keterkaitan yang erat antara teori manajemen memori, arsitektur prosesor, dan implementasi nyata pada sistem operasi.

5.2 Saran

Untuk pengembangan praktikum selanjutnya, disarankan agar simulasi dilakukan dengan beban kerja yang lebih bervariasi dan mendekati kondisi sistem nyata, seperti pola akses non-acak dan jumlah proses yang lebih besar. Selain itu, pengukuran performa dapat ditingkatkan dengan memanfaatkan tools profiling sistem untuk memperoleh data yang lebih akurat. Pengembangan simulasi juga dapat diperluas dengan menambahkan mekanisme seperti copy-on-write, demand paging, atau penggunaan TLB secara eksplisit.

5.3 Kendala yang Dihadapi

Selama pelaksanaan praktikum, kendala utama yang dihadapi adalah keterbatasan observasi langsung terhadap beberapa komponen perangkat keras, seperti ukuran dan detail TLB, yang tidak selalu dapat diakses tanpa utilitas tambahan. Selain itu, simulasi

yang dilakukan bersifat abstraksi sehingga tidak sepenuhnya mencerminkan kompleksitas sistem operasi nyata. Tantangan lain yang dihadapi adalah memastikan konsistensi hasil eksperimen akibat adanya mekanisme randomisasi alamat memori seperti ASLR, yang menyebabkan variasi alamat pada setiap eksekusi program.

DAFTAR PUSTAKA

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Hoboken, NJ: John Wiley & Sons.

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson Education.

LAMPIRAN

1. TUGAS 1

```
senjaramadhani@LAPTOP-823QCJCF:~/praktikum-memory/architecture$ nano enhanced_memory_layout.c
senjaramadhani@LAPTOP-823QCJCF:~/praktikum-memory/architecture$ gcc enhanced_memory_layout.c -o enhanced_memory_layout
senjaramadhani@LAPTOP-823QCJCF:~/praktikum-memory/architecture$ ./enhanced_memory_layout
=== ENHANCED MEMORY LAYOUT ===

1. TEXT SEGMENT:
main() address : 0x57e0b7295326
func1() address: 0x57e0b7295209
func2() address: 0x57e0b729525d

2. DATA SEGMENT:
global_var address : 0x57e0b7298010
static_array[0] address : 0x57e0b7298020
string_literal address : 0x57e0b72960d9

3. HEAP:
dynamic_array[0] address: 0x57e0e2e426b0
malloc block 1 address : 0x57e0e2e426d0
malloc block 2 address : 0x57e0e2e42700

4. STACK (RECURSIVE CALLS):
Level 0 local_var address: 0x7ffdd2453744
Level 1 local_var address: 0x7ffdd2453714
Level 2 local_var address: 0x7ffdd24536e4

Process PID: 1222
Check memory map with:
cat /proc/1222/maps
```

source code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
/* ----- Global & Static (Data Segment) ----- */
```

```
int global_var = 100;
```

```
static int static_array[5] = {1, 2, 3, 4, 5};
```

```
/* ----- Function Declarations (Text Segment) ----- */
```

```
void func1();
```

```
void func2();
```

```
void recursive_func(int level, int max);
```

```
/* ----- Functions ----- */
```

```

void func1() {
    int local_func1 = 10;
    printf("func1 local var address : %p\n", (void *)&local_func1);
}

```

```

void func2() {
    int local_func2 = 20;
    printf("func2 local var address : %p\n", (void *)&local_func2);
}

```

```

void recursive_func(int level, int max) {
    int local_recursive = level;
    printf("Level %d local_var address: %p\n",
        level, (void *)&local_recursive);

    if (level < max) {
        recursive_func(level + 1, max);
    }
}

```

```

int main() {
    printf("=== ENHANCED MEMORY LAYOUT ===\n\n");

    /* ----- TEXT SEGMENT ----- */
    printf("1. TEXT SEGMENT:\n");
}

```

```

printf("main() address : %p\n", (void *)&main);
printf("func1() address: %p\n", (void *)&func1);
printf("func2() address: %p\n\n", (void *)&func2);

/* ----- DATA SEGMENT ----- */

const char *string_literal = "Hello Memory Layout";

printf("2. DATA SEGMENT:\n");
printf("global_var address    : %p\n", (void *)&global_var);
printf("static_array[0] address : %p\n", (void *)&static_array[0]);
printf("string_literal address  : %p\n\n", (void *)string_literal);

/* ----- HEAP ----- */

printf("3. HEAP:\n");
int *dynamic_array = (int *)malloc(5 * sizeof(int));
int *block1 = (int *)malloc(10 * sizeof(int));
int *block2 = (int *)malloc(20 * sizeof(int));

printf("dynamic_array[0] address: %p\n", (void *)&dynamic_array[0]);
printf("malloc block 1 address  : %p\n", (void *)block1);
printf("malloc block 2 address  : %p\n\n", (void *)block2);

/* ----- STACK & RECURSION ----- */

printf("4. STACK (RECURSIVE CALLS):\n");
recursive_func(0, 2);

```

```

printf("\n");

/* ----- PID FOR /proc/maps ----- */

printf("Process PID: %d\n", getpid());
printf("Check memory map with:\n");
printf("cat /proc/%d/maps\n", getpid());

/* ----- CLEANUP ----- */

free(dynamic_array);

free(block1);

free(block2);

return 0;
}

```

2. TUGAS 2

```

senjaramadhani@LAPTOP-823QCJCF:~/praktikum-memory/architecture$ nano memory_allocator.c
senjaramadhani@LAPTOP-823QCJCF:~/praktikum-memory/architecture$ gcc memory_allocator.c -o memory_allocator
senjaramadhani@LAPTOP-823QCJCF:~/praktikum-memory/architecture$ ./memory_allocator

=== First-Fit ===
Execution Time: 0.000004 seconds
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 3
External Fragmentation : 0.40

=== Best-Fit ===
Execution Time: 0.000003 seconds
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 3
External Fragmentation : 0.40

=== Worst-Fit ===
Execution Time: 0.000024 seconds
Total Free Space      : 170
Largest Free Block    : 100
Number of Free Holes  : 3
External Fragmentation : 0.41

=== Next-Fit ===
Execution Time: 0.000002 seconds
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 3
External Fragmentation : 0.40

```


Source code:

```
#include <stdio.h>

#include <time.h>

#define MEMORY_SIZE 1000

#define MAX_BLOCKS 100

/* ----- Struktur Memory Block ----- */
typedef struct {
    int start;
    int size;
    int free;
    int pid;
} Block;

Block memory[MAX_BLOCKS];
int block_count;

/* ----- Next-Fit Pointer ----- */
int last_alloc_pos = 0;

/* ----- Inisialisasi Memory ----- */
void init_memory() {
    block_count = 1;
    memory[0].start = 0;
```

```

memory[0].size = MEMORY_SIZE;
memory[0].free = 1;
memory[0].pid = -1;
last_alloc_pos = 0;
}

/* ----- Split Block ----- */
void split_block(int index, int pid, int size) {
    Block new_block;
    new_block.start = memory[index].start + size;
    new_block.size = memory[index].size - size;
    new_block.free = 1;
    new_block.pid = -1;

    memory[index].size = size;
    memory[index].free = 0;
    memory[index].pid = pid;

    for (int i = block_count; i > index + 1; i--) {
        memory[i] = memory[i - 1];
    }
    memory[index + 1] = new_block;
    block_count++;
}

```

```

/* ----- First-Fit ----- */
int first_fit(int pid, int size) {
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            split_block(i, pid, size);
            return 1;
        }
    }
    return 0;
}

/* ----- Best-Fit ----- */
int best_fit(int pid, int size) {
    int best = -1;
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (best == -1 || memory[i].size < memory[best].size) {
                best = i;
            }
        }
    }
    if (best != -1) {
        split_block(best, pid, size);
        return 1;
    }
}

```

```

    return 0;
}

/* ----- Worst-Fit ----- */
int worst_fit(int pid, int size) {
    int worst = -1;
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (worst == -1 || memory[i].size > memory[worst].size) {
                worst = i;
            }
        }
    }
    if (worst != -1) {
        split_block(worst, pid, size);
        return 1;
    }
    return 0;
}

/* ----- Next-Fit ----- */
int next_fit(int pid, int size) {
    int i = last_alloc_pos;
    int scanned = 0;

```

```

while (scanned < block_count) {
    if (memory[i].free && memory[i].size >= size) {
        split_block(i, pid, size);
        last_alloc_pos = i;
        return 1;
    }
    i = (i + 1) % block_count;
    scanned++;
}
return 0;
}

```

/* ----- Deallocate ----- */

```

void deallocate(int pid) {
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].free && memory[i].pid == pid) {
            memory[i].free = 1;
            memory[i].pid = -1;
        }
    }
}

```

/* ----- Fragmentation ----- */

```

void calculate_fragmentation() {
    int total_free = 0;

```

```

int largest_free = 0;
int holes = 0;

for (int i = 0; i < block_count; i++) {
    if (memory[i].free) {
        total_free += memory[i].size;
        holes++;
        if (memory[i].size > largest_free) {
            largest_free = memory[i].size;
        }
    }
}

double ext_frag = 0.0;
if (total_free > 0) {
    ext_frag = 1.0 - ((double)largest_free / total_free);
}

printf("Total Free Space      : %d\n", total_free);
printf("Largest Free Block    : %d\n", largest_free);
printf("Number of Free Holes   : %d\n", holes);
printf("External Fragmentation : %.2f\n", ext_frag);
}

/* ----- Benchmark ----- */

```

```

void benchmark_test(int (*alloc)(int, int), const char *name) {
    init_memory();
    clock_t start = clock();

    alloc(1,100);
    alloc(2,200);
    alloc(3,300);
    alloc(4,250);
    alloc(5,150);

    deallocate(2);
    deallocate(4);

    alloc(6,180);
    alloc(7,220);
    alloc(8,100);

    clock_t end = clock();
    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("\n=== %s ===\n", name);
    printf("Execution Time: %f seconds\n", time_spent);
    calculate_fragmentation();
}

```

```

/* ----- MAIN ----- */

int main() {

    benchmark_test(first_fit, "First-Fit");

    benchmark_test(best_fit, "Best-Fit");

    benchmark_test(worst_fit, "Worst-Fit");

    benchmark_test(next_fit, "Next-Fit");

    return 0;

}

```

3. TUGAS 3

```

senjaramadhani@LAPTOP-823QCJCF:~$ nano paging_simulator.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc paging_simulator.c -o paging_simulator
senjaramadhani@LAPTOP-823QCJCF:~$ ./paging_simulator

=== FIFO SIMULATION ===
Access page 7 | Frames: 7 - - -
Access page 0 | Frames: 7 0 - -
Access page 1 | Frames: 7 0 1 -
Access page 2 | Frames: 7 0 1 2
Access page 0 | Frames: 7 0 1 2
Access page 3 | Frames: 3 0 1 2
Access page 0 | Frames: 3 0 1 2
Access page 4 | Frames: 3 4 1 2
Access page 2 | Frames: 3 4 1 2
Access page 3 | Frames: 3 4 1 2
Access page 0 | Frames: 3 4 0 2
Access page 3 | Frames: 3 4 0 2
Access page 2 | Frames: 3 4 0 2
Access page 1 | Frames: 3 4 0 1
Access page 2 | Frames: 2 4 0 1
Access page 0 | Frames: 2 4 0 1
Access page 1 | Frames: 2 4 0 1
Access page 7 | Frames: 2 7 0 1
Access page 0 | Frames: 2 7 0 1
Access page 1 | Frames: 2 7 0 1

STATISTICS
Total Accesses : 20
Page Faults    : 10
Page Hits      : 10
Hit Ratio      : 0.50
Disk Writes    : 0

```



```

=== LRU SIMULATION ===
Access page 7 | Frames: 7 - - -
Access page 0 | Frames: 7 0 - -
Access page 1 | Frames: 7 0 1 -
Access page 2 | Frames: 7 0 1 2
Access page 0 | Frames: 7 0 1 2
Access page 3 | Frames: 3 0 1 2
Access page 0 | Frames: 3 0 1 2
Access page 4 | Frames: 3 0 4 2
Access page 2 | Frames: 3 0 4 2
Access page 3 | Frames: 3 0 4 2
Access page 0 | Frames: 3 0 4 2
Access page 3 | Frames: 3 0 4 2
Access page 2 | Frames: 3 0 4 2
Access page 1 | Frames: 3 0 1 2
Access page 2 | Frames: 3 0 1 2
Access page 0 | Frames: 3 0 1 2
Access page 1 | Frames: 3 0 1 2
Access page 7 | Frames: 7 0 1 2
Access page 0 | Frames: 7 0 1 2
Access page 1 | Frames: 7 0 1 2

STATISTICS
Total Accesses : 20
Page Faults    : 8
Page Hits      : 12
Hit Ratio      : 0.60
Disk Writes    : 0
senjaramadhani@LAPTOP-823QCJCF:~$ |

```

Source code:

```

#include <stdio.h>
#include <stdbool.h>

#define FRAMES 4
#define PAGES 50
#define REF_LEN 20

typedef struct {
    int total_accesses;
    int page_faults;
    int page_hits;
    float hit_ratio;
    int disk_writes;
} PageStatistics;

/* GLOBAL */
int frames[FRAMES];

```

```

int fifo_index = 0;
int time_counter = 0;
int last_used[PAGES];

/* INIT MEMORY */
void init_memory() {
    for (int i = 0; i < FRAMES; i++)
        frames[i] = -1;

    for (int i = 0; i < PAGES; i++)
        last_used[i] = -1;

    fifo_index = 0;
    time_counter = 0;
}

/* CHECK PAGE HIT */
bool page_in_memory(int page) {
    for (int i = 0; i < FRAMES; i++)
        if (frames[i] == page)
            return true;
    return false;
}

/* FIFO PAGE REPLACEMENT */
void fifo_page_replacement(int page) {
    frames[fifo_index] = page;
    fifo_index = (fifo_index + 1) % FRAMES;
}

/* LRU PAGE REPLACEMENT */
void lru_page_replacement(int page) {
    int lru_page = frames[0];
    int lru_index = 0;

    for (int i = 1; i < FRAMES; i++) {

```

```

        if (last_used[frames[i]] < last_used[lru_page]) {
            lru_page = frames[i];
            lru_index = i;
        }
    }
    frames[lru_index] = page;
}

/* DISPLAY MEMORY */
void display_frames() {
    for (int i = 0; i < FRAMES; i++) {
        if (frames[i] == -1)
            printf(" - ");
        else
            printf(" %d ", frames[i]);
    }
    printf("\n");
}

/* DISPLAY STATISTICS */
void display_statistics(PageStatistics *stats) {
    stats->hit_ratio = (float)stats->page_hits / stats->total_accesses;

    printf("\nSTATISTICS\n");
    printf("Total Accesses : %d\n", stats->total_accesses);
    printf("Page Faults   : %d\n", stats->page_faults);
    printf("Page Hits      : %d\n", stats->page_hits);
    printf("Hit Ratio      : %.2f\n", stats->hit_ratio);
    printf("Disk Writes    : %d\n", stats->disk_writes);
}

/* SIMULATION */
void simulate(int reference[], int length, bool use_lru) {
    PageStatistics stats = {0, 0, 0, 0.0, 0};
    init_memory();

```

```
printf(use_lru ? "\n=== LRU SIMULATION ===\n" : "\n=== FIFO
SIMULATION ===\n");
```

```
for (int i = 0; i < length; i++) {
    int page = reference[i];
    stats.total_accesses++;
```

```
printf("Access page %d | Frames:", page);
```

```
if (page_in_memory(page)) {
    stats.page_hits++;
} else {
    stats.page_faults++;
```

```
    bool placed = false;
    for (int j = 0; j < FRAMES; j++) {
        if (frames[j] == -1) {
            frames[j] = page;
            placed = true;
            break;
        }
    }
}
```

```
if (!placed) {
    if (use_lru)
        lru_page_replacement(page);
    else
        fifo_page_replacement(page);
}
}
```

```
last_used[page] = time_counter++;
display_frames();
}
```

```
display_statistics(&stats);
```

```

}

int main() {
    int reference[REF_LEN] = {
        7, 0, 1, 2, 0, 3, 0, 4, 2, 3,
        0, 3, 2, 1, 2, 0, 1, 7, 0, 1
    };

    simulate(reference, REF_LEN, false); // FIFO
    simulate(reference, REF_LEN, true); // LRU

    return 0;
}

```

4. TUGAS 4

```

senjaramadhani@LAPTOP-823QCJCF:~$ nano multilevel_pagetable.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc multilevel_pagetable.c -o multilevel_pagetable
senjaramadhani@LAPTOP-823QCJCF:~$ ./multilevel_pagetable
Virtual Address : 0x123456789abc
Physical Address: 0x5abc

Memory Overhead
L1 entries used: 1
L2 entries used: 1

Inverted Page Table Search Result: Frame 3
senjaramadhani@LAPTOP-823QCJCF:~$ |

```

Source code:

```

#include <stdio.h>

#include <stdlib.h>

#define L1_BITS 16

#define L2_BITS 8

#define L3_BITS 8

#define OFFSET_BITS 12

#define NUM_FRAMES 64

```

```
typedef struct {
    int frame_number;

    int valid;

    int read, write, execute;
} L3_Entry;
```

```
typedef struct {
    L3_Entry *entries;

    int valid;
} L2_Entry;
```

```
typedef struct {
    L2_Entry *entries;

    int valid;
} L1_Entry;
```

```
/* ROOT PAGE TABLE */
```

```
L1_Entry *page_directory = NULL;
```

```
/* THREE LEVEL TRANSLATION */
```

```
int translate_3level(unsigned long virtual_addr) {
    unsigned long l1 = (virtual_addr >> (L2_BITS + L3_BITS +
    OFFSET_BITS)) & ((1 << L1_BITS) - 1);

    unsigned long l2 = (virtual_addr >> (L3_BITS + OFFSET_BITS)) & ((1 <<
    L2_BITS) - 1);
```

```
    unsigned long l3 = (virtual_addr >> OFFSET_BITS) & ((1 << L3_BITS) - 1);
```

```
    unsigned long offset = virtual_addr & ((1 << OFFSET_BITS) - 1);
```

```
    if (!page_directory[l1].valid) return -1;
```

```
    if (!page_directory[l1].entries[l2].valid) return -1;
```

```
    if (!page_directory[l1].entries[l2].entries[l3].valid) return -1;
```

```
    int frame = page_directory[l1].entries[l2].entries[l3].frame_number;
```

```
    return (frame << OFFSET_BITS) | offset;
```

```
}
```

```
void allocate_3level(unsigned long virtual_addr, int frame) {
```

```
    unsigned long l1 = (virtual_addr >> (L2_BITS + L3_BITS + OFFSET_BITS)) & ((1 << L1_BITS) - 1);
```

```
    unsigned long l2 = (virtual_addr >> (L3_BITS + OFFSET_BITS)) & ((1 << L2_BITS) - 1);
```

```
    unsigned long l3 = (virtual_addr >> OFFSET_BITS) & ((1 << L3_BITS) - 1);
```

```
    if (!page_directory[l1].valid) {
```

```
        page_directory[l1].entries = calloc(1 << L2_BITS, sizeof(L2_Entry));
```

```
        page_directory[l1].valid = 1;
```

```
    }
```

```
    if (!page_directory[l1].entries[l2].valid) {
```

```

    page_directory[l1].entries[l2].entries = calloc(1 << L3_BITS,
sizeof(L3_Entry));

```

```

    page_directory[l1].entries[l2].valid = 1;
}

```

```

L3_Entry *entry = &page_directory[l1].entries[l2].entries[l3];
entry->frame_number = frame;
entry->valid = 1;
entry->read = entry->write = entry->execute = 1;
}

```

```

void calculate_memory_overhead() {
    int l1_used = 0, l2_used = 0, l3_used = 0;

    for (int i = 0; i < (1 << L1_BITS); i++) {
        if (page_directory[i].valid) {
            l1_used++;
            for (int j = 0; j < (1 << L2_BITS); j++) {
                if (page_directory[i].entries[j].valid)
                    l2_used++;
            }
        }
    }
}

```

```

printf("\nMemory Overhead\n");

```



```

    printf("L1 entries used: %d\n", l1_used);
    printf("L2 entries used: %d\n", l2_used);
}

/* INVERTED PAGE TABLE */
typedef struct {
    int process_id;
    int page_number;
    int valid;
} InvertedPageEntry;

InvertedPageEntry inverted_table[NUM_FRAMES];

int search_inverted_table(int pid, int page_num) {
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (inverted_table[i].valid &&
            inverted_table[i].process_id == pid &&
            inverted_table[i].page_number == page_num)
            return i;
    }
    return -1;
}

void insert_inverted_table(int pid, int page_num, int frame) {
    inverted_table[frame].process_id = pid;

```

```

    inverted_table[frame].page_number = page_num;
    inverted_table[frame].valid = 1;
}

int main() {
    page_directory = calloc(1 << L1_BITS, sizeof(L1_Entry));

    unsigned long vaddr = 0x123456789ABC;
    allocate_3level(vaddr, 5);

    int phys = translate_3level(vaddr);
    printf("Virtual Address : 0x%lx\n", vaddr);
    printf("Physical Address: 0x%x\n", phys);

    calculate_memory_overhead();

    insert_inverted_table(1, 10, 3);
    int frame = search_inverted_table(1, 10);
    printf("\nInverted Page Table Search Result: Frame %d\n", frame);

    return 0;
}

```

5. TUGAS 5

```

FIFO Strategy
=== SIMULATION START ===
Time 0: Access P8
Swap IN : P8 (75 MB)
Time 1: Access P6
Swap IN : P6 (50 MB)
Time 2: Access P9
Swap IN : P9 (72 MB)
Time 3: Access P6
Time 4: Access P8
Time 5: Access P6
Time 6: Access P1
Swap IN : P1 (95 MB)
Time 7: Access P5
Swap IN : P5 (134 MB)
Time 8: Access P9
Time 9: Access P3
Swap IN : P3 (117 MB)
Time 10: Access P5
Time 11: Access P9
Time 12: Access P3
Time 13: Access P9
Time 27: Access P2
Time 28: Access P5
Time 29: Access P5
Time 30: Access P1
Time 31: Access P2
Time 32: Access P10
Time 33: Access P1
Time 34: Access P9
Time 35: Access P9
Time 36: Access P9
Time 37: Access P9
Time 38: Access P5
Time 39: Access P7
Swap OUT: P1 (95 MB)
Swap IN : P7 (130 MB)
Time 40: Access P4
Time 41: Access P9
Time 42: Access P7
Time 43: Access P6
Time 44: Access P10
Time 45: Access P5
Time 67: Access P1
Swap OUT: P2 (166 MB)
Swap IN : P1 (95 MB)
Time 68: Access P8
Time 69: Access P3
Time 70: Access P9
Time 71: Access P1
Time 72: Access P2
Swap OUT: P3 (117 MB)
Swap IN : P2 (166 MB)
Time 73: Access P5
Time 74: Access P8
Time 75: Access P3
Swap OUT: P4 (66 MB)
Swap IN : P3 (117 MB)
Time 76: Access P1
Time 77: Access P5
Time 78: Access P4
Swap OUT: P5 (134 MB)
Swap IN : P4 (66 MB)
Time 79: Access P8
Time 80: Access P1
Time 96: Access P5
Swap OUT: P6 (50 MB)
Swap OUT: P8 (75 MB)
Swap IN : P5 (134 MB)
Time 97: Access P4
Time 98: Access P8
Swap OUT: P9 (72 MB)
Swap IN : P8 (75 MB)
Time 99: Access P5
=== RESULT ===
P1 swap count: 1
P2 swap count: 1
P3 swap count: 1
P4 swap count: 1
P5 swap count: 1
P6 swap count: 1
P7 swap count: 0
P8 swap count: 1
P9 swap count: 1
P10 swap count: 0
LRU Strategy
=== SIMULATION START ===
Time 0: Access P7
Time 1: Access P6
Swap IN : P6 (50 MB)
Time 90: Access P6
Time 91: Access P1
Time 92: Access P10
Time 93: Access P4
Time 94: Access P5
Time 95: Access P6
Time 96: Access P3
Time 97: Access P8
Time 98: Access P4
Time 99: Access P5
=== RESULT ===
P1 swap count: 1
P2 swap count: 1
P3 swap count: 1
P4 swap count: 1
P5 swap count: 1
P6 swap count: 1
P7 swap count: 0
P8 swap count: 1
P9 swap count: 1
P10 swap count: 0
Priority Strategy
=== SIMULATION START ===
Time 0: Access P10
Time 86: Access P10
Time 87: Access P1
Time 88: Access P7
Time 89: Access P7
Time 90: Access P3
Time 91: Access P5
Time 92: Access P2
Time 93: Access P6
Time 94: Access P4
Time 95: Access P1
Time 96: Access P4
Time 97: Access P2
Time 98: Access P3
Time 99: Access P10
=== RESULT ===
P1 swap count: 1
P2 swap count: 1
P3 swap count: 1
P4 swap count: 1
P5 swap count: 1
P6 swap count: 1
P7 swap count: 0
P8 swap count: 1
P9 swap count: 1
P10 swap count: 0

```

```

senjaramadhani@LAPTOP-823QCJCF:~$ nano swapping_simulator.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc swapping_simulator.c -o swapping_simulator
senjaramadhani@LAPTOP-823QCJCF:~$ ./swapping_simulator

FIFO Strategy
=== SIMULATION START ===

Time 0: Access P8
Swap IN : P8 (75 MB)

Time 1: Access P6
Swap IN : P6 (50 MB)

Time 2: Access P9
Swap IN : P9 (72 MB)

Time 3: Access P6

Time 4: Access P8

Time 5: Access P6

Time 6: Access P1
Swap IN : P1 (95 MB)

Time 7: Access P5
Swap IN : P5 (134 MB)

Time 8: Access P9

Time 9: Access P3
Swap IN : P3 (117 MB)

Time 10: Access P5

Time 11: Access P9

Time 12: Access P3

Time 13: Access P9

Time 14: Access P8

Time 15: Access P10
Swap IN : P10 (176 MB)

Time 16: Access P3

Time 17: Access P10

Time 18: Access P8

Time 19: Access P2
Swap IN : P2 (166 MB)

Time 20: Access P3

Time 21: Access P1

Time 22: Access P4
Swap IN : P4 (66 MB)

Time 23: Access P2

Time 24: Access P10

Time 25: Access P1

Time 26: Access P8

Time 27: Access P2

```

Source code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#define MAX_PROCESS 10
```

```
#define MEMORY_SIZE 1024
```

```
#define SIM_TIME 100
```

```

typedef struct {
    int pid;
    int size;
    int priority;
    int in_memory;
    int swap_count;
    time_t last_access;
    time_t load_time;
} Process;

/* FUNCTION PROTOTYPES */
void swap_out(Process *p);
void swap_in(Process *p, int strategy);
void select_victim(int strategy);
int fifo_victim();
int lru_victim();
int priority_victim();

Process processes[MAX_PROCESS];
int used_memory = 0;

/* SWAP OUT */
void swap_out(Process *p) {
    if (p->in_memory) {
        p->in_memory = 0;
        used_memory -= p->size;
        p->swap_count++;
        printf("Swap OUT: P%d (%d MB)\n", p->pid, p->size);
    }
}

/* SWAP IN */

```

```

void swap_in(Process *p, int strategy) {
    while (used_memory + p->size > MEMORY_SIZE) {
        select_victim(strategy);
    }
    p->in_memory = 1;
    p->load_time = time(NULL);
    used_memory += p->size;
    printf("Swap IN : P%d (%d MB)\n", p->pid, p->size);
}

/* FIFO */
int fifo_victim() {
    int idx = -1;
    time_t oldest = time(NULL);

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].load_time < oldest) {
            oldest = processes[i].load_time;
            idx = i;
        }
    }
    return idx;
}

/* LRU */
int lru_victim() {
    int idx = -1;
    time_t oldest = time(NULL);

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].last_access < oldest) {
            oldest = processes[i].last_access;

```

```

        idx = i;
    }
}
return idx;
}

/* PRIORITY */
int priority_victim() {
    int idx = -1;
    int lowest = 999;

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].priority < lowest) {
            lowest = processes[i].priority;
            idx = i;
        }
    }
    return idx;
}

/* SELECT VICTIM */
void select_victim(int strategy) {
    int idx = -1;

    if (strategy == 0) idx = fifo_victim();
    else if (strategy == 1) idx = lru_victim();
    else if (strategy == 2) idx = priority_victim();

    if (idx != -1)
        swap_out(&processes[idx]);
}

```

```

/* INIT */
void init_processes() {
    srand(time(NULL));
    for (int i = 0; i < MAX_PROCESS; i++) {
        processes[i].pid = i + 1;
        processes[i].size = 50 + rand() % 151;
        processes[i].priority = 1 + rand() % 5;
        processes[i].in_memory = 0;
        processes[i].swap_count = 0;
        processes[i].last_access = time(NULL);
    }
}

/* SIMULATION */
void simulate(int strategy) {
    used_memory = 0;
    printf("\n=== SIMULATION START ===\n");

    for (int t = 0; t < SIM_TIME; t++) {
        int pid = rand() % MAX_PROCESS;
        Process *p = &processes[pid];

        printf("\nTime %d: Access P%d\n", t, p->pid);
        p->last_access = time(NULL);

        if (!p->in_memory) {
            swap_in(p, strategy);
        }
    }

    printf("\n=== RESULT ===\n");
    for (int i = 0; i < MAX_PROCESS; i++) {

```



```

        printf("P%d swap count: %d\n",
               processes[i].pid,
               processes[i].swap_count);
    }
}

int main() {
    init_processes();

    printf("\nFIFO Strategy");
    simulate(0);

    printf("\nLRU Strategy");
    simulate(1);

    printf("\nPriority Strategy");
    simulate(2);

    return 0;
}

```

6. TUGAS 6

```

senjaramadhani@LAPTOP-823QCJCF:~$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
CPU(s): 16
On-line CPU(s) list: 0-15
Model name: AMD Ryzen 7 7435HS
CPU family: 25
Model: 68
Thread(s) per core: 2
Core(s) per socket: 8
NUMA node0 CPU(s): 0-15
senjaramadhani@LAPTOP-823QCJCF:~$ getconf PAGE_SIZE
4096
senjaramadhani@LAPTOP-823QCJCF:~$ cpuid | grep -i tlb
Command 'cpuid' not found, but can be installed with:
sudo apt install cpuid
senjaramadhani@LAPTOP-823QCJCF:~$ lscpu | grep -i cache
L1d cache: 256 KiB (8 instances)
L1i cache: 256 KiB (8 instances)
L2 cache: 4 MiB (8 instances)
L3 cache: 16 MiB (1 instance)

```

```

senjaramadhani@LAPTOP-823QCJCF:~$ free -h
Mem: 3.7Gi 381Mi 3.2Gi
Swap: 1.0Gi 0 1.0Gi
senjaramadhani@LAPTOP-823QCJCF:~$ vmstat 1 5
procs-----memory-----swap-----io-----system-----cpu-----
 r b swpd free buff cache si so bi bo in cs us sy id wa st
0 0 0 3325968 7640 192968 0 0 6 0 16 16 0 0 100 0 0
0 0 0 3325968 7640 193016 0 0 0 0 270 250 0 0 100 0 0
0 0 0 3325968 7640 193016 0 0 0 0 239 228 0 0 100 0 0
0 0 0 3325968 7640 193016 0 0 0 0 259 234 0 0 100 0 0
0 0 0 3325712 7640 193016 0 0 0 0 320 298 0 0 100 0 0
senjaramadhani@LAPTOP-823QCJCF:~$ cat /proc/self/maps
55baf7d70000-55baf7d7a000 r--p 00000000 00:30 1501 /usr/bin/cat
55baf7d7a000-55baf7d7e000 r-xp 00002000 00:30 1501 /usr/bin/cat
55baf7d7e000-55baf7d80000 r--p 00006000 00:30 1501 /usr/bin/cat
55baf7d80000-55baf7d81000 r--p 00007000 00:30 1501 /usr/bin/cat
55baf7d81000-55baf7d82000 rw-p 00008000 00:30 1501 /usr/bin/cat
55bb28342000-55bb28363000 rw-p 00000000 00:00 0 [heap]
794803fde000-794804000000 rw-p 00000000 00:00 0
794804000000-794804028000 r--p 00000000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
794804028000-7948041bd000 r-xp 00028000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
7948041bd000-794804215000 r--p 001bd000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
794804215000-794804216000 r--p 00215000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
794804216000-79480421a000 r--p 00215000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
79480421a000-79480421c000 rw-p 00219000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
79480421c000-794804229000 rw-p 00000000 00:00 0
794804240000-79480429b000 r--p 00000000 00:30 14612 /usr/lib/locale/C.utf8/LC_CTYPE
79480429b000-79480429c000 r--p 00000000 00:30 16421 /usr/lib/locale/C.utf8/LC_NUMERIC
79480429c000-79480429d000 r--p 00000000 00:30 16424 /usr/lib/locale/C.utf8/LC_TIME
79480429d000-79480429e000 r--p 00000000 00:30 14597 /usr/lib/locale/C.utf8/LC_COLLATE
79480429e000-79480429f000 r--p 00000000 00:30 16419 /usr/lib/locale/C.utf8/LC_MONETARY
79480429f000-7948042a0000 r--p 00000000 00:30 14615 /usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES
7948042a0000-7948042a1000 r--p 00000000 00:30 16422 /usr/lib/locale/C.utf8/LC_PAPER
7948042a1000-7948042a2000 r--p 00000000 00:30 16420 /usr/lib/locale/C.utf8/LC_NAME
7948042a2000-7948042a3000 r--p 00000000 00:30 14596 /usr/lib/locale/C.utf8/LC_ADDRESS
7948042a3000-7948042a4000 r--p 00000000 00:30 16423 /usr/lib/locale/C.utf8/LC_TELEPHONE
7948042a4000-7948042a7000 rw-p 00000000 00:00 0
7948042a7000-7948042a8000 r--p 00000000 00:30 14614 /usr/lib/locale/C.utf8/LC_MEASUREMENT
55baf7d70000-55baf7d7a000 r--p 00000000 00:30 1501 /usr/bin/cat
55baf7d7a000-55baf7d7e000 r-xp 00002000 00:30 1501 /usr/bin/cat
55baf7d7e000-55baf7d80000 r--p 00006000 00:30 1501 /usr/bin/cat
55baf7d80000-55baf7d81000 r--p 00007000 00:30 1501 /usr/bin/cat
55baf7d81000-55baf7d82000 rw-p 00008000 00:30 1501 /usr/bin/cat
55bb28342000-55bb28363000 rw-p 00000000 00:00 0 [heap]
794803fde000-794804000000 rw-p 00000000 00:00 0
794804000000-794804028000 r--p 00000000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
794804028000-7948041bd000 r-xp 00028000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
7948041bd000-794804215000 r--p 001bd000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
794804215000-794804216000 r--p 00215000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
794804216000-79480421a000 r--p 00215000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
79480421a000-79480421c000 rw-p 00219000 00:30 50697 /usr/lib/x86_64-linux-gnu/libc.so.6
79480421c000-794804229000 rw-p 00000000 00:00 0
794804240000-79480429b000 r--p 00000000 00:30 14612 /usr/lib/locale/C.utf8/LC_CTYPE
79480429b000-79480429c000 r--p 00000000 00:30 16421 /usr/lib/locale/C.utf8/LC_NUMERIC
79480429c000-79480429d000 r--p 00000000 00:30 16424 /usr/lib/locale/C.utf8/LC_TIME
79480429d000-79480429e000 r--p 00000000 00:30 14597 /usr/lib/locale/C.utf8/LC_COLLATE
79480429e000-79480429f000 r--p 00000000 00:30 16419 /usr/lib/locale/C.utf8/LC_MONETARY
79480429f000-7948042a0000 r--p 00000000 00:30 14615 /usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES
7948042a0000-7948042a1000 r--p 00000000 00:30 16422 /usr/lib/locale/C.utf8/LC_PAPER
7948042a1000-7948042a2000 r--p 00000000 00:30 16420 /usr/lib/locale/C.utf8/LC_NAME
7948042a2000-7948042a3000 r--p 00000000 00:30 14596 /usr/lib/locale/C.utf8/LC_ADDRESS
7948042a3000-7948042a4000 r--p 00000000 00:30 16423 /usr/lib/locale/C.utf8/LC_TELEPHONE
7948042a4000-7948042a7000 rw-p 00000000 00:00 0
7948042a7000-7948042a8000 r--p 00000000 00:30 14614 /usr/lib/locale/C.utf8/LC_MEASUREMENT
7948042a8000-7948042af000 r--p 00000000 00:30 50978 /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
7948042af000-7948042b1000 rw-p 00000000 00:00 0
7948042b1000-7948042b3000 r--p 00000000 00:30 50693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7948042b3000-7948042dd000 r-xp 00028000 00:30 50693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7948042dd000-7948042e8000 r--p 0021c000 00:30 50693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7948042e8000-7948042e9000 r--p 00000000 00:30 14613 /usr/lib/locale/C.utf8/LC_IDENTIFICATION
7948042e9000-7948042eb000 r--p 00037000 00:30 50693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7948042eb000-7948042ed000 rw-p 00039000 00:30 50693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffe93a2d000-7ffe93a4e000 rw-p 00000000 00:00 0 [stack]
7ffe93a9a000-7ffe93a9e000 r--p 00000000 00:00 0 [vvar]
7ffe93a9e000-7ffe93aaa0000 r-xp 00000000 00:00 0 [vdso]
senjaramadhani@LAPTOP-823QCJCF:~$ swapon --show
NAME TYPE SIZE USED PRIO
/dev/sdc partition 1G 0B -2
senjaramadhani@LAPTOP-823QCJCF:~$

```